

13 Guia Maestra

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

13 Guía Maestra	4
title: "Cartridge & Cloud — Guía Maestra" subtitle: "Manual operativo integral para dirigir, implementar, validar, publicar, transferir y gobernar el proyecto" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: es-ES version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE"	5
0. Cómo utilizar esta Guía Maestra	5
Regla de lectura	5
Ampliación de la regla de lectura para los documentos 17-34	6
Resultados que esta guía exige	6
1. Propósito, alcance y límites	6
Incluido	6
No incluido como compromiso actual	7
2. Principios operativos del proyecto	7
3. Jerarquía documental y autoridad	7
Reglas de resolución	8
4. Estado actual del desarrollo y de la consolidación documental	8
Trabajo técnico obligatorio inmediato	8
Secuencia documental autorizada	9
Regla de paralelismo	9
5. Cómo iniciar una sesión de trabajo	9
Entrada mínima	9
Secuencia de apertura	9
Salida de la apertura	9
6. Cómo revisar dashboard, riesgos y gates	10
Regla de revisión	10
7. Cómo elegir la siguiente tarea	10
Desempates	10
8. Cómo limitar WIP	11
Señales de WIP tóxico	11
9. Cómo preparar un sprint	11
10. Cómo definir un sprint charter	11
Contenido obligatorio	11
Ejemplo aplicado a Sprint 16	12
11. Cómo dividir trabajo en paquetes	12
Criterios de división	12
12. Cómo estimar sin falsa precisión	12
13. Cómo registrar dependencias y bloqueos	13
Bloqueo frente a riesgo	13
14. Cómo implementar una feature	13
Definition of Done de feature	14
15. Cómo corregir un defecto	14
Ejemplo: clic de UI mueve al jugador	14
16. Cómo gestionar deuda técnica	14
Deuda que no debe ocultarse	15
17. Cómo cambiar arquitectura	15
18. Cómo crear y aprobar un ADR	15
Cuándo es obligatorio	15
Plantilla mínima	16
Aprobación	16
19. Cómo modificar el modelo de datos	16
Preguntas obligatorias	16
20. Cómo evolucionar schemas	16
Estado conocido	17
21. Cómo trabajar con save/load	17
Pipeline de guardado	17

Pipeline de carga	17
Pruebas mínimas	17
22. Cómo implementar operaciones atómicas	18
Ámbitos actuales	18
23. Cómo integrar UI e input	18
Antipatrones	18
24. Cómo trabajar con escenas	19
Reglas	19
25. Cómo autorar StoreInitial	19
Separación de contenido	19
26. Cómo importar assets	20
Regla de reimportación	20
27. Cómo crear prefabs y catálogos	20
Prefab	20
Catálogo	20
Registro runtime	21
28. Cómo integrar personajes	21
29. Cómo integrar audio y VFX	21
Validación	21
30. Cómo revisar C#	22
Cierre de revisión	22
31. Cómo ejecutar EditMode y PlayMode	22
Baseline de referencia	22
32. Cómo ejecutar Golden Path	22
Fallo del Golden Path	23
33. Cómo realizar una campaña de siete días	23
Evidencia	23
34. Cómo hacer profiling	23
No optimizar a ciegas	24
35. Cómo hacer balance	24
Límites	24
36. Cómo registrar defectos	24
Calidad del reporte	25
37. Cómo aplicar severidades S0-S4	25
Contadores distintos	25
38. Cómo decidir si un defecto bloquea	25
Regla H6	25
39. Cómo preparar una build	25
Tipos de build	26
40. Cómo revisar Player.log	26
Criterio	26
41. Cómo archivar una build	26
Retención	27
42. Cómo preparar una candidata de hito	27
43. Cómo cerrar un sprint	27
Condiciones	27
Procedimiento	27
44. Cómo aprobar un hito	28
45. Cómo actualizar producción	28
No usar el Excel como	28
46. Cómo actualizar trazabilidad	28
47. Cómo actualizar documentación	29
Definition of Done documental	29
48. Cómo congelar una baseline	29
Baseline no significa perfección	30
49. Cómo crear un handoff	30
Contenido mínimo	30
Criterio de calidad	30
50. Cómo recuperar una máquina o proyecto	30

Limpieza segura	31
51. Cómo actualizar Unity o paquetes	31
Prohibición	31
52. Cómo gestionar secretos, seguridad y licencias	31
Secretos	31
Licencias	31
Archivos de usuario	32
53. Cómo controlar scope creep	32
Pregunta decisiva	32
54. Cómo tratar sistemas futuros	32
Regla de diseño futuro	32
54.1. Cierre formal de Sprint 16	32
55. Cómo abrir Sprint 17	33
Workstreams	33
56. Cómo aprobar H6	33
PASS with Accepted Debt	34
57. Cómo preparar la fase post-H6	34
58. Checklists operativos	34
Checklist diaria	34
Checklist feature	34
Checklist escena	35
Checklist build	35
Checklist cierre	35
59. Plantillas reutilizables	36
Plantilla de tarea	36
Plantilla de defecto	36
Plantilla de revisión de gate	36
Plantilla de handoff	36
60. Glosario, problemas frecuentes y mantenimiento de la guía	37
Glosario	37
Problemas frecuentes	37
Mantenimiento	37
Criterios de aceptación de la Guía Maestra	37
Playbooks end-to-end de referencia	38
Definition of Ready por tipo de trabajo	41
Definition of Done por tipo de trabajo	42
Taxonomía de evidencia	42
Cadencia de mantenimiento operativo	42
Heurísticas de decisión y señales de parada	43
Matriz de impacto documental por cambio	44
Protocolo de revisión final de una sesión	44
Manual operativo por sistema del vertical slice	44
Paquetes de evidencia por gate actual	47
Manual de escalado de incidencias	48
Preguntas de revisión por disciplina	49
Ejemplo de sesión completa aplicada a S16-WP-004	50
Ejemplo de sesión completa aplicada a un cambio de save	50
Catálogo de antipatrones y corrección esperada	51
Estrategia de rollback y puntos de no retorno	51
Protocolo de retrospectiva de sprint o hito	52
Producción individual: disciplina sin burocracia	53
Criterios para pausar, cancelar o replanificar trabajo	53
Revisión de coherencia antes de publicar una baseline	54
Matriz de responsabilidad operativa por artefacto	54
Protocolo de continuidad entre sesiones largas	55
61. Cómo utilizar las autoridades especializadas 17-30	55
61.1. Secuencia común de consulta	55
61.2. Mapa de uso	55
62. Cómo operar 31_H6_and_Release_Readiness_Checklist.xlsx	56

62.1. Regla de estado	56
62.2. Flujo de ejecución	56
62.3. Separación de gates	56
63. Cómo operar 32_Documentation_Baseline_Manifest.xlsx	56
63.1. Ciclo candidato-final	56
63.2. Qué no hacer	57
64. Cómo operar 33_Project_Risk_Register_and_Business_Continuity_Plan.xlsx	57
64.1. Revisión de un riesgo	57
64.2. Continuidad	57
65. Cómo operar 34_Final_Handoff_and_Project_Operations_Manual.md	57
66. Cómo ejecutar la consolidación documental v2.0	57
66.1. Regla de adopción	58
67. Cómo autorizar claims y comunicación pública	58
67.1. Matriz mínima	58
68. Cómo integrar accesibilidad e inclusión en el trabajo diario	59
69. Cómo medir y optimizar rendimiento	59
69.1. Registro mínimo de profiling	59
70. Cómo coordinar legal, privacidad, seguridad y continuidad	59
71. Cómo validar el paquete documental final	60
71.1. Controles automáticos	60
71.2. Controles humanos	60
72. Rutas de onboarding por rol	60
72.1. Producción y dirección	60
72.2. Ingeniería	60
72.3. Arte, audio y UI	60
72.4. QA y release	60
72.5. Marketing, comunidad y soporte	60
73. Cómo conservar y usar la documentación histórica	61
74. Matriz de decisión sobre qué artefacto actualizar	61
75. Criterios de aceptación de esta Guía Maestra	61
Anexo A. Inventario de la baseline documental utilizada	61
A.1. Interpretación del inventario	62
Anexo B. Evolución de las Guías Maestras históricas	62
Anexo C. Fuentes de consolidación	62
Anexo D. Próxima revisión obligatoria	62
66. Operación de la remediación Sprint17_Phase1	63
Estado operativo vigente tras W0	63
Decisiones formalizadas	63
Pruebas de caracterización previas a las olas	63
Secuencia operativa obligatoria	64
Regla de cierre y no propagación	64
67. Decisión final e implementación de modificaciones diferidas S17	64

13 Guía Maestra

Metadatos normativos v2.0

document_version: 2.0

project_version: 0.0.26

last_reviewed: 2026-07-17

status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE

lifecycle: LIVING DOCUMENT

title: "Cartridge & Cloud — Guía Maestra" subtitle: "Manual operativo integral para dirigir, implementar, validar, publicar, transferir y gobernar el proyecto" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: es-ES version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE"

Proyecto: Cartridge & Cloud **Estudio / autor:** VRM Games / Blas Luis Rocha González
Plataforma inicial: PC / Steam **Motor:** Unity 6.3 LTS 6000.3.18f1 **Render pipeline:** URP 17.3.0 **Lenguaje:** C# 9.0 / netstandard2.1 **Versión de aplicación observada:** 0.0.26
Estado global: Sprints 0-15 CLOSED / PASS; Sprint 16 COMPLETED / PASS; Sprint 17 APPROVED / IMPLEMENTED / VALIDATED / CLOSED; H6 APPROVED / IMPLEMENTED / VALIDATED / CLOSED; Release NOT OPEN HISTORICAL EVIDENCE — infraestructura automatizada retirada: **Baseline automatizada de referencia:** 1215 EditMode + 70 PlayMode = 1285 PASS **Trabajo técnico inmediato:** apertura y ejecución controlada de Sprint 17: balance, siete días, rendimiento, localización, QA y candidata H6
Trabajo documental inmediato: adoptar esta Guía, actualizar 21, 23, 31, 33 y 34, generar manifiesto 32 candidato y regenerar 16 **Baseline documental:** conjunto completo 00-34; documentos 12 y 13 regenerados; 14 y 15 actualizados; consolidación v2.0 en curso.

Esta guía explica **cómo trabajar**. No sustituye a los documentos especializados que definen **qué debe ser cierto**. Ante una duda de alcance, diseño, arquitectura, datos, UX, QA, entorno, build o estado operativo, debe consultarse la fuente competente indicada por el Project Binder.

Las cifras de backlog, pruebas, defectos, riesgos, builds y cobertura son fotografías operativas. Deben actualizarse en los Excel correspondientes y no convertirse en compromisos de calendario por mera extrapolación.

0. Cómo utilizar esta Guía Maestra

La Guía Maestra convierte el paquete documental en un sistema de trabajo repetible. Su objetivo no es ofrecer una única receta rígida, sino impedir que una sesión de desarrollo comience sin contexto, que una implementación se cierre sin evidencia o que una decisión importante quede repartida entre chats, memoria personal y archivos inconexos.

El lector debe usarla como manual de navegación operativa. Cada capítulo describe entradas, pasos, controles, salidas y errores que deben evitarse. Cuando un procedimiento menciona un requisito, una invariante, una decisión o un gate, la autoridad final permanece en el documento especializado correspondiente.

Regla de lectura

Pregunta	Fuente principal	Uso de esta guía
¿Qué juego estamos construyendo?	00 y 01	Aplicar límites y convertir intención en trabajo
¿Qué debe incluir H6?	02	Ejecutar el procedimiento de validación y cierre
¿Cómo se organiza el código?	03 y 09	Aplicar workflow de implementación y revisión
¿Qué datos e invariantes existen?	04	Aplicar migración, atomicidad y persistencia
¿Cómo debe comportarse la experiencia?	05	Aplicar integración UI/input y pruebas UX
¿Qué toca ahora?	06 y 12	Seleccionar tarea, limitar WIP y registrar avances
¿Cómo se prueba?	07 y 08	Diseñar, ejecutar y registrar evidencia
¿Cómo se prepara Unity?	10	Restaurar, configurar e importar correctamente
¿Cómo se genera una build?	11	Preflight, build, smoke, log y archivo
¿Por qué cambió algo?	13	Crear cambio, ADR, impacto y cierre

Pregunta	Fuente principal	Uso de esta guía
¿Dónde está la verdad?	14	Resolver autoridad y contradicciones

Ampliación de la regla de lectura para los documentos 17-34

Pregunta	Fuente principal	Uso de esta guía
¿Cómo debe verse el juego y cómo se acepta visualmente?	17 y 19	Preparar briefs, integrar assets
¿Cómo debe sonar y qué equivalentes accesibles necesita?	18 y 29	Implementar audio, mezcla
¿Qué valores económicos y contenido inicial se utilizan?	20 y 21	Autorizar datos, IDs, balance
¿Cómo se localiza y se valida ES/EN?	22	Preparar strings, pseudolocalización
¿Puede usarse, acreditarse o distribuirse un asset?	23	Verificar licencia, proveniencia
¿Cómo se prepara Steam y una release?	24, 25 y 31	Ejecutar gates separados de Steam
¿Qué datos se pueden recopilar?	26	Aplicar minimización, retención
¿Cómo se protegen secretos y se responde a incidentes?	27 y 33	Aplicar controles, playbooks
¿Qué puede comunicarse públicamente?	28	Autorizar claims, canales y redes
¿Qué barreras deben eliminarse?	29	Diseñar, implementar y probar
¿Cómo se mide y optimiza rendimiento?	30	Definir escenario, presupuesto
¿Cómo se decide H6 o release?	31	Ejecutar controles, evidenciar
¿Qué contiene y cómo se congela la baseline?	32	Inventariar, hashear, validar
¿Qué puede interrumpir el proyecto?	33	Gestionar riesgos, backups, rollback
¿Cómo se retoma o transfiere el proyecto?	34	Aplicar onboarding, handoff

Resultados que esta guía exige

- Cada sesión comienza con baseline y objetivo explícitos.
- Cada tarea tiene criterio de aceptación, evidencia y fuente.
- Cada cambio importante deja rastro en producción y trazabilidad.
- Cada cierre separa implementación, validación, build y documentación.
- Cada gate puede reproducirse sin depender de memoria informal.

1. Propósito, alcance y límites

Esta guía cubre el ciclo completo desde la apertura de una sesión hasta la congelación de una baseline. Incluye trabajo de producto, ingeniería, escena, contenido, QA, builds y documentación. Está escrita para una producción individual, pero conserva disciplina suficiente para que otra persona pueda auditar, continuar o revisar el proyecto.

No reemplaza la creatividad ni obliga a producir documentos por cada acción trivial. Sí obliga a formalizar aquello que puede romper una baseline, alterar el alcance, cambiar una invariante, modificar un schema, introducir una dependencia, afectar una build o invalidar evidencia previa.

Incluido

- selección y preparación de trabajo
- implementación de features y correcciones
- gestión de arquitectura y datos
- autoría de escenas y contenido

- QA automatizado y manual
- builds, logs y artefactos
- producción, trazabilidad y documentación
- scope control, seguridad y recuperación

No incluido como compromiso actual

- fechas públicas de lanzamiento
- Steamworks ya integrado
- Steam Cloud operativo
- empleados, investigación o puestos como trabajo abierto
- online, publishing o desarrollo interno como alcance de H6
- infraestructura o plataforma digital como feature aprobada

2. Principios operativos del proyecto

Principio	Aplicación
La evidencia prevalece sobre la suposición	Una afirmación de que algo funciona no equivale a la verdad.
El alcance se abre por gates	Un sistema futuro no se inicia porque sea atractivo.
La UI comunica; la simulación decide	La presentación solicita acciones y muestra resultados.
Las mutaciones críticas son atómicas	Inventario, checkout, ledger y save no deben ser concurrentes.
La persistencia forma parte del sistema	Una feature persistente no está terminada si no se puede guardar.
Escena autorada y runtime tienen responsabilidades distintas	La escena fija se autoriza manualmente; el runtime se genera automáticamente.
Un test verde no aprueba la presentación	Sprint 16 exige revisión visual, recorrido manual y pruebas de integración.
Un porcentaje no es una fecha	El progreso de filas del Excel no representa el tiempo restante.
La historia no se borra	Las sustituciones se registran; los documentos se archivan.
Un cambio importante actualiza toda la cadena	Diseño, técnica, datos, QA, producción, builds y documentación.

3. Jerarquía documental y autoridad

La jerarquía no se interpreta como una lista lineal en la que el último documento siempre gana. Cada fuente tiene autoridad dentro de su ámbito, y una decisión solo puede propagarse correctamente si respeta la constitución del producto, el contrato de aceptación, la evidencia observada y el control de cambios.

Nivel	Documentos
T0 — Constitución	00
T1 — Contratos maestros	01-11
T2 — Gobierno y navegación	12-16
T3 — Autoridades especializadas	17-30
T4 — Cierre operativo	31-34
E — Evidencia	Código, escenas, ProjectSettings, tests, logs, builds, capturas y hashes
H — Historia	Baselines v0.3-v0.6, ADR, sprints, handoffs, manifests y fuentes de autoridad

Una fuente histórica puede ser más detallada que una revisión posterior. Cuando ese detalle no contradice la autoridad actual, se conserva como antecedente operativo; cuando sí la contradice, se registra la sustitución en 13 y se mantiene el original inmutable.

Reglas de resolución

1. Identificar si el texto describe intención, estado real, evidencia, historia o visión.
2. Aplicar el documento más específico dentro del ámbito correcto.
3. Comprobar versión, fecha, estado y decisiones ADR relacionadas.
4. Contrastar con código, ProjectSettings, escena, test o build cuando se discuta implementación.
5. Clasificar la diferencia como defecto, deuda, error documental, cambio aceptado o registro histórico.
6. Registrar la decisión en el control de cambios antes de alterar varias fuentes.

4. Estado actual del desarrollo y de la consolidación documental

La fotografía vigente separa cuatro planos. No deben reducirse a un único porcentaje:

1. **Producto y código:** Sprints 0-15 tienen cierre histórico CLOSED / PASS; Sprint 16 está COMPLETED / PASS y StoreInitial dispone de integración y aceptación representativa.
2. **Validación:** H6 no se ha ejecutado contra la matriz completa del documento 31; los estados NOT RUN no son fallos, pero bloquean una aprobación.
3. **Release:** Steam, Coming Soon, demo, Playtest, RC y lanzamiento son gates posteriores y separados. Ninguno está abierto por el mero hecho de existir planes 24, 25 y 28.
4. **Documentación:** los documentos 00-34 existen, pero la baseline final todavía requiere actualizaciones dirigidas, auditoría, manifest definitivo, checksums, informe de validación y ZIP inmutable.

Dimensión	Estado vigente
Aplicación	0.0.26
HISTORICAL EVIDENCE — infraestructura automatizada retirada:	Automatización histórica
Backlog maestro	612 elementos
Work packages	224
Sprint 16	COMPLETED / PASS
Sprint 17	APPROVED / IMPLEMENTED / VALIDATED
H6	APPROVED / IMPLEMENTED / VALIDATED
Release readiness	NOT OPEN / NOT RUN
Riesgos	180 registrados; 174 fuera de tolerancia
Historia preservada	745 archivos; 643 hashes únicos; 98 grupos
Documentos actuales	35 presentes
Consolidación	4/22 antes de adoptar esta guía; 5/22 al

Trabajo técnico obligatorio inmediato

1. Congelar y registrar la baseline de entrada de Sprint 17.
2. Abrir formalmente Sprint 17 sin introducir sistemas mayores nuevos.
3. Ejecutar balance y sesión completa de siete días.
4. Ejecutar perfilado, carga objetivo y soak test en build.
5. Completar localización mínima ES/EN o registrar una excepción formal del gate.
6. Preparar los perfiles Windows_QA y Windows_H6_Candidate.
7. Consolidar evidencia de QA, rendimiento, persistencia, localización y logs.
8. Ejecutar Sprint 17 y preparar la candidata H6 sin declarar todavía la Vertical Slice aprobada.

Secuencia documental autorizada

1. Adoptar los documentos regenerados 12 y 13.
2. Adoptar el Binder 14 y esta Guía 15.
3. Actualizar 21 y 23.
4. Actualizar 31 y, de forma dirigida, 33 y 34.
5. Generar el manifiesto 32 candidato.
6. Regenerar 16, registrar y resolver hallazgos.
7. Regenerar 16 final y 32 definitivo.
8. Generar `PACKAGE_MANIFEST.md`, `CHECKSUMS_SHA256.txt` y `DOCUMENTATION_VALIDATION_REPORT.md`.
9. Crear, extraer y verificar el ZIP inmutable.
10. Asociar la baseline a commit y tag; después ejecutar H6 y continuidad.

Regla de paralelismo

La consolidación documental no autoriza por sí sola cambios de código, apertura de Sprint 17 ni comunicación pública. Cuando se trabaje en paralelo, cada flujo debe mantener su propia baseline, owner, evidencia y cierre. Un cambio técnico que afecte documentos debe registrarse en 13; un cambio documental que altere requisitos debe volver al owner del contrato correspondiente.

5. Cómo iniciar una sesión de trabajo

Entrada mínima

- working copy identificada
- rama y commit conocidos
- Unity y paquetes correctos
- dashboard de producción revisado
- Sprint 16 o tarea autorizada
- bloqueos y gate visibles

Secuencia de apertura

1. Abrir GitHub Desktop o la herramienta de control de versiones y revisar rama, cambios locales y upstream.
2. Confirmar que no existen modificaciones desconocidas, archivos generados o metadatos inesperados.
3. Leer el estado ejecutivo de `12_Excel_Maestro_de_Produccion.xlsx` y los avisos de `13_Trazabilidad_y_Control_de_Cambios.xlsx`.
4. Identificar una única tarea principal y, como máximo, una tarea auxiliar que no compita por el mismo riesgo.
5. Leer el requisito, ADR y criterio de aceptación vinculados.
6. Definir qué evidencia se producirá antes de abrir Unity o editar código.
7. Comprobar que la tarea no invade Sprint 17, sistemas futuros o un gate no abierto.
8. Registrar el inicio si la tarea cambia de estado o consume capacidad relevante.

Salida de la apertura

La sesión debe comenzar con una frase operativa concreta: “Voy a completar S16-WP-004, usando `StoreInitialSceneContext`, y la evidencia será una validación de referencias, captura del layout,

PlayMode dirigido y diff revisado". Una intención vaga como "avanzar la escena" no es suficiente para evaluar cierre o bloqueo.

6. Cómo revisar dashboard, riesgos y gates

El dashboard resume, pero no reemplaza las filas que lo alimentan. Una cifra anómala debe investigarse en la hoja de origen antes de decidir trabajo.

Señal	Dónde profundizar	Acción
Blocked aumenta	05_Sprint_Activo / 07_Dependencias	resolver predecesor o registrar decisión
Riesgo Critical	08_Riesgos	activar mitigación y definir trigger
S0/S1 > 0	09_Defectos_Deuda y QA Matrix	separar defecto QA de deuda; decidir gate
Gate FAIL	12_QA_Gates	abrir criterios y evidencia faltante
Build PENDING	13_Builds_Releases	ver preflight, SHA, escenas y Player.log
Auditoría FAIL	13_Trazabilidad... / 17_Auditoria	corregir integridad, no ocultar alerta

Regla de revisión

La revisión debe responder tres preguntas: qué está bloqueando el resultado actual, qué evidencia falta y cuál es la mínima acción que reduce el riesgo sin abrir alcance nuevo. No debe convertirse en una replanificación total diaria.

7. Cómo elegir la siguiente tarea

La selección de tareas usa prioridad, dependencia, gate, riesgo y capacidad; no solo preferencia personal. El algoritmo evita trabajar en una tarea visual mientras una referencia técnica imprescindible sigue sin definirse, o comenzar estabilización cuando la escena representativa aún no está conectada.

1. Filtrar por sprint activo y estado Open, In Progress o Blocked cuyo bloqueo pueda resolverse.
2. Excluir tareas que dependan de resultados no cerrados.
3. Priorizar P0, después P1; usar P2 solo si desbloquea evidencia o reduce riesgo.
4. Favorecer tareas que cierran un gate o permiten ejecutar varias pruebas posteriores.
5. Comprobar que existe un criterio de aceptación verificable.
6. Comprobar que el tamaño cabe en una sesión o dividir el paquete.
7. Evitar empezar más trabajo si hay una tarea casi cerrada pendiente de evidencia o documentación.

Desempates

Situación	Preferencia
Dos P0 disponibles	la que desbloquee más dependencias críticas
Feature frente a bug S1	bug S1
Código frente a evidencia pendiente	evidencia si la implementación ya está completa
Contenido frente a integración	integración funcional antes de pulido adicional
Refactor frente a H6	solo refactor necesario para cerrar defecto o riesgo

8. Cómo limitar WIP

En una producción individual, WIP excesivo oculta el coste de contexto. La regla recomendada es una tarea principal activa, una tarea auxiliar de baja interferencia y ningún tercer frente salvo incidencia bloqueante.

- No mantener simultáneamente una migración de save, una escena, un cambio de paquetes y una build candidata.
- No abrir un refactor “mientras tanto” si el bloqueo necesita una decisión concreta.
- Cerrar evidencia y trazabilidad antes de declarar libre la capacidad.
- Dividir tareas que mezclen autoría visual, código, QA y documentación en paquetes secuenciales.

Señales de WIP tóxico

- más de tres ramas conceptuales en la working copy
- tests rotos por cambios no relacionados
- archivos de varios sprints modificados sin ADR
- tareas In Progress sin último paso definido
- necesidad de recordar qué parte era temporal

9. Cómo preparar un sprint

Un sprint se prepara cuando el gate anterior permite abrirlo y existe una baseline estable. No se abre para agrupar ideas. Debe tener un resultado verificable, límites, dependencias, riesgos, evidencia y política de cierre.

1. Confirmar estado del sprint anterior y del hito asociado.
2. Congelar baseline de entrada: versión, SHA, suites, build y known issues.
3. Definir objetivo en términos de resultado jugable o técnico observable.
4. Enumerar inclusiones y exclusiones.
5. Dividir en work packages ordenados por dependencia.
6. Asignar criterios de aceptación y evidencia a cada paquete.
7. Identificar riesgos, mitigaciones y triggers.
8. Preparar casos de prueba y datos antes de implementar.
9. Reservar capacidad para integración, defectos, build y documentación.
10. Publicar charter, estado inicial y entradas en producción/trazabilidad.

10. Cómo definir un sprint charter

Contenido obligatorio

Campo	Contenido
Objetivo	resultado único que justifica el sprint
Baseline	versión, SHA, tests y build de entrada

Campo	Contenido
Incluido	features, contenido, correcciones y evidencia
Excluido	trabajo explícitamente fuera del sprint
Gates	condiciones para abrir, integrar y cerrar
Riesgos	probabilidad, impacto, mitigación y contingencia
DoD	implementación, QA, build, documentación y trazabilidad
Handoff	qué debe poder continuar otra sesión/persona

Ejemplo aplicado a Sprint 16

Sprint 16 se dividió en Fase 1 funcional y Fase 2 representativa. Esa separación impidió sustituir blackout antes de tener Golden Path, QA, build y Player externo. La lección general es que una fase visual no debe ocultar dependencias funcionales ni convertir assets en criterio de arquitectura.

11. Cómo dividir trabajo en paquetes

Un paquete debe producir una salida verificable y no mezclar responsabilidades que se validan de forma distinta. Para StoreInitial, crear la escena, autorar arquitectura, conectar referencias, desactivar fallback y aprobar una build son paquetes diferentes.

Paquete correcto	Salida	Evidencia
Crear StoreInitial.unity	escena controlada con GUID y contenido base	diff, carga y referencias
Autorar arquitectura	suelo, muros, entrada y zonas	capturas y recorrido
Crear SceneContext	referencias obligatorias explícitas	validación y test
Conectar runtime	sistemas operan escena autorada	Golden Path dirigido
Desactivar procedural	sin shell duplicado	test/captura comparativa
Build post-integración	EXE reproducible	record, log y checksum

Criterios de división

- diferente owner conceptual
- diferente tipo de evidencia
- riesgo independiente
- posibilidad de rollback separado
- dependencia que debe ser explícita
- tamaño que excede una sesión razonable

12. Cómo estimar sin falsa precisión

La estimación sirve para detectar desproporciones y reservar capacidad, no para producir una fecha contractual. Se recomienda usar puntos relativos o categorías pequeñas, medianas y grandes, acompañadas por riesgo y dependencia.

Factor	Pregunta
Novedad	¿existe un patrón cerrado o se investiga una solución?
Integración	¿cuántos sistemas y documentos toca?
Evidencia	¿requiere build, campaña o revisión visual?

Factor	Pregunta
Rollback	¿puede revertirse sin migrar datos?
Contenido	¿depende de asset externo o autoría manual?
Incertidumbre	¿hay más de una arquitectura plausible?

Nunca convertir el 22,22 % registrado para Sprint 16 en una predicción de tiempo restante. Ocho paquetes cerrados pueden ser menores o mayores que los pendientes; además, integración y QA suelen concentrar riesgo al final.

13. Cómo registrar dependencias y bloqueos

Una dependencia expresa una condición objetiva. Un bloqueo expresa que la condición no se cumple y evita avanzar. No deben usarse para describir mera preferencia de orden.

1. Asignar ID de predecesor y sucesor.
2. Indicar tipo: finish-to-start, gate, dato, contenido o decisión.
3. Explicar por qué la dependencia existe.
4. Definir qué evidencia la satisface.
5. Marcar el bloqueo en la tarea, no solo en una nota.
6. Revisar el bloqueo cuando cambia el predecesor.
7. Cerrar o sustituir la relación cuando deja de ser válida.

Bloqueo frente a riesgo

Un bloqueo ya impide el trabajo; un riesgo podría impedirlo. Por ejemplo, una puerta sin orientación aprobada bloquea integración. La posibilidad de que un FBX futuro cambie escala es un riesgo. Ambos requieren registro distinto.

14. Cómo implementar una feature

1. Localizar requisito, sistema, sprint, ADR y casos de prueba.
2. Definir precondition, comando o interacción, mutación autoritativa y resultado observable.
3. Revisar boundaries de assemblies y dependencias permitidas.
4. Diseñar datos configurables y evitar hardcode no justificado.
5. Implementar primero reglas puras e invariantes; después adaptadores Unity y presentación.
6. Añadir manejo de error y resultado explícito.
7. Añadir o actualizar tests al mismo tiempo que el código.
8. Integrar en escena mediante composition root o contexto explícito.
9. Ejecutar pruebas dirigidas y después regresión del área.
10. Revisar diff, logs, allocations y persistencia cuando aplique.
11. Actualizar producción, trazabilidad y documentos impactados.

Definition of Done de feature

- compila sin warnings nuevos no aceptados
- criterios satisfechos
- tests nominales, límite y negativos
- save/load cubierto si persiste
- UI no posee lógica autoritativa
- errores visibles y registrables
- sin búsqueda global innecesaria
- evidencia vinculada
- documentación y cambio actualizados

15. Cómo corregir un defecto

1. Registrar reproducción, entorno, versión/build y severidad.
2. Distinguir síntoma, causa y daño potencial.
3. Crear test de regresión cuando el fallo sea automatizable.
4. Identificar la invariante o contrato roto.
5. Aplicar la mínima corrección que restaure el contrato sin ocultar el problema.
6. No modificar el test para acomodar un comportamiento incorrecto.
7. Ejecutar prueba dirigida, sistema afectado y regresión proporcional.
8. Comprobar persistencia, idempotencia o scene state si el defecto afecta mutaciones.
9. Cerrar solo con evidencia y build cuando el gate la exija.

Ejemplo: clic de UI mueve al jugador

El síntoma es movimiento al hacer clic en Operations. La causa probable es que el input de mundo procesa el mismo evento que el EventSystem. La corrección debe establecer prioridad exclusiva de UI, probar clic sobre elementos interactivos y no interactivos, y verificar que el mundo sigue respondiendo cuando el puntero no está sobre UI.

16. Cómo gestionar deuda técnica

La deuda no es cualquier código imperfecto. Es una desviación conocida respecto al estándar o arquitectura objetivo, con coste o riesgo futuro. Debe registrarse cuando se acepta deliberadamente no resolverla ahora.

Campo	Contenido
ID	estable y único
Descripción	qué desviación existe
Motivo	por qué se acepta temporalmente
Impacto	qué puede romper o encarecer
Trigger	cuándo deja de ser aceptable
Objetivo	sprint o fase de revisión
Mitigación	cómo se limita mientras permanece

Campo	Contenido
Evidencia	qué demuestra que no bloquea el gate actual

Deuda que no debe ocultarse

- dos modelos de persistencia coexistiendo
- placement dinámico incompleto en schema integrado
- ausencia de editorconfig/analyzers
- BuildCommand y CI todavía futuros
- tablas ES/EN no creadas
- resolución objetivo no congelada
- colisión histórica de ADR ya normalizada

17. Cómo cambiar arquitectura

Un cambio arquitectónico modifica responsabilidades, dependencias, lifecycle, persistencia, composición o contratos públicos. No debe entrar como “refactor” sin impacto documental.

1. Describir problema actual y evidencia de que el patrón existente no basta.
2. Enumerar alternativas, incluida mantener el diseño actual.
3. Analizar impacto en Domain, Application, Infrastructure, Presentation, escenas, save y tests.
4. Crear ADR con decisión y consecuencias.
5. Planificar migración incremental y rollback.
6. Actualizar TDD, Modelo de Datos y Coding Standards si corresponde.
7. Implementar tras tests de caracterización cuando se sustituye comportamiento existente.
8. Cerrar con comparación antes/después y evidencia de no regresión.

18. Cómo crear y aprobar un ADR

Cuándo es obligatorio

- cambiar Unity, paquete o baseline
- alterar assemblies o dependencias
- cambiar ownership de escena o composition root
- modificar schema o estrategia de persistencia
- cambiar atomicidad/idempotencia
- introducir servicio externo
- cambiar Build Profile o política de release
- aceptar cambio mayor de alcance

Plantilla mínima

ADR-NNNN – Título

```
**Estado:** Proposed / Accepted / Superseded
**Fecha:** YYYY-MM-DD
**Contexto:** problema y restricciones
**Decisión:** opción elegida
**Alternativas:** opciones rechazadas y motivos
**Consecuencias positivas:** ...
**Consecuencias negativas / deuda:** ...
**Migración y rollback:** ...
**Validación:** tests, build, métricas o revisión
**Documentos impactados:** ...
```

Aprobación

Un ADR queda aceptado cuando la decisión es comprensible, el impacto está registrado y existe un plan verificable. No necesita que toda la implementación esté completa, pero sí debe distinguir claramente decisión de ejecución.

19. Cómo modificar el modelo de datos

1. Identificar entidad, value object, record o snapshot afectado.
2. Definir invariantes antes de campos y serialización.
3. Distinguir estado autoritativo de proyección UI o cache.
4. Analizar igualdad, hash, orden y unidades.
5. Revisar compatibilidad binaria/documental y save schema.
6. Definir creación válida y rechazo de estados imposibles.
7. Actualizar Modelo de Datos y TDD.
8. Añadir tests de constructor, límites, igualdad, serialización y migración.

Preguntas obligatorias

- ¿quién crea el dato?
- ¿quién puede mutarlo?
- ¿qué unidad usa?
- ¿puede ser nulo?
- ¿se persiste?
- ¿cómo se migra?
- ¿cómo se compara?
- ¿qué pasa con una versión futura?

20. Cómo evolucionar schemas

El schema es un contrato de persistencia. Incrementarlo requiere una razón concreta, un lector consciente de versión y una política de compatibilidad.

1. Documentar versión origen y destino.

2. Definir si la lectura es backward compatible, requiere migración o se rechaza.
3. No sobrescribir el único backup durante una migración.
4. Validar el snapshot completo antes de mutar runtime.
5. Conservar identificadores estables y unidades exactas.
6. Probar primario antiguo, primario actual, versión futura y archivo corrupto.
7. Registrar cambio, ADR, build y evidencia.

Estado conocido

El proyecto ha usado GameSessionSnapshot schema 1 y IntegratedGameStateSnapshot schema 2. La persistencia completa de placement dinámico permanece como deuda futura y no debe confundirse con la arquitectura fija autorada de StoreInitial.

21. Cómo trabajar con save/load

Pipeline de guardado

Capturar estado autoritativo
→ construir snapshot válido
→ serializar de forma determinista
→ escribir temporal
→ validar temporal
→ preservar backup
→ reemplazar primario atómicamente
→ registrar resultado

Pipeline de carga

Leer primario
→ validar envelope/schema/checksum
→ si falla, evaluar backup
→ construir estado candidato
→ validar invariantes
→ commit de restauración en segunda fase
→ publicar resultado o error explícito

Pruebas mínimas

- round trip
- tres slots
- replace/delete
- primario corrupto
- backup válido
- permisos insuficientes
- temporal interrumpido
- versión futura
- equivalencia de inventario, reservas, cola, economía y día

22. Cómo implementar operaciones atómicas

Una operación atómica no deja cambios parciales si falla. El patrón general es preflight, reserva o validación, commit único e idempotencia cuando puede repetirse un comando.

1. Reunir todas las precondiciones sin mutar.
2. Calcular cantidades, dinero, capacidad y referencias.
3. Rechazar con resultado explícito si algo falla.
4. Aplicar mutaciones en un orden que preserve invariantes o dentro de un commit lógico.
5. Registrar un identificador idempotente cuando el comando pueda reintentarse.
6. Emitir eventos solo después del commit.
7. Probar fallo en cada frontera y repetición del mismo comando.

Ámbitos actuales

Sistema	Invariante
Inventario	la suma de unidades se conserva
Reservas	stock disponible nunca incluye dos veces lo reservado
Checkout	producto y dinero se confirman una sola vez
Ledger	un evento económico idempotente no duplica asiento
Save	primario nunca queda parcialmente escrito

23. Cómo integrar UI e input

La UI y el mundo comparten dispositivos, pero no deben procesar la misma intención. La prioridad depende del contexto de escena, modal y foco.

1. Definir action maps y contexto activo.
2. Comprobar EventSystem y pointer-over-UI antes de acciones de mundo.
3. Procesar modal, panel y escape por prioridad.
4. Mantener lógica autoritativa fuera de la vista.
5. Presentar estados de carga, vacío, error y confirmación.
6. Probar teclado/ratón, resoluciones y foco tras abrir/cerrar paneles.
7. Validar que la supresión de input UI no rompe clics válidos del mundo.

Antipatrones

- Button modifica inventario directamente
- Update polling duplicado en varias vistas
- GameObject.Find para descubrir servicios
- cerrar modal y mover jugador con el mismo clic
- usar color como única señal
- mostrar claves de localización crudas

24. Cómo trabajar con escenas

Una escena contiene composición y referencias, no debe convertirse en contenedor accidental de servicios globales duplicados. Bootstrap posee el arranque; MainMenu gestiona slots y navegación; Store es la base histórica funcional; StoreInitial es la escena representativa objetivo; TestLab es desarrollo y no debe distribuirse.

Reglas

- crear y guardar escenas desde Unity
- preservar GUID y .meta
- usar roots con responsabilidad clara
- mantener managers globales fuera del prefab de entorno
- usar referencias serializadas o contextos explícitos
- no inferir arquitectura por nombres, bounds o jerarquía importada
- validar build order/profile después de cambios

25. Cómo autorar StoreInitial

StoreInitial debe conservar la funcionalidad de Store mientras sustituye la presentación procedural o placeholder por una escena manual coherente. El tamaño objetivo es aproximadamente 10 × 15 m, con grid lógico 20 × 30 de 0,5 m.

1. Crear StoreInitial.unity como copia controlada de la escena funcional, preservando referencias necesarias.
2. Crear StoreInitialEnvironment.prefab para arquitectura y mobiliario fijo, sin HUD, cámara, EventSystem, persistencia o managers globales.
3. Autorar suelo, muros, entrada, almacén, recepción y checkout con medidas y colliders coherentes.
4. Configurar puerta automática con transforms y referencias explícitas; no inferir hojas por nombres.
5. Colocar mobiliario inicial bajo un root separado del placement dinámico.
6. Añadir anchors y roots técnicos con nombres estables y propósito documentado.
7. Crear StoreInitialSceneContext y validar todas las referencias obligatorias antes de iniciar gameplay.
8. Conectar sistemas existentes: inventario, recepción, displays, clientes, cola, checkout, día, UI y save.
9. Mantener fallback procedural aislado hasta demostrar que no es necesario.
10. Desactivar el shell procedural mediante configuración explícita y comprobar ausencia de duplicados.
11. Ejecutar validación visual, accesibilidad, Golden Path, suites y build externa.

Separación de contenido

Root	Contenido	Persistencia
FixedArchitecture	suelo, muros, entrada, zonas	escena/prefab; no snapshot dinámico
InitialFurniture	mobiliario de arranque aprobado	escena o estado inicial controlado
DynamicPlacement	objetos colocados por jugador	snapshot de placement futuro/completo
TechnicalAnchors	spawn, delivery, queue, checkout, navigation	referencias técnicas

26. Cómo importar assets

1. Confirmar origen, licencia, versión y archivo fuente.
2. Importar a la carpeta correcta sin mover .meta existentes.
3. Validar escala, ejes, pivote, jerarquía y nombres.
4. Configurar materiales, texturas, compresión y read/write según necesidad.
5. Configurar colliders; evitar MeshCollider costoso sin justificación.
6. Configurar LOD y comprobar bounds.
7. Crear prefab de proyecto separado del FBX importado.
8. Asignar ID y catálogo cuando el runtime lo necesita.
9. Validar en una escena de prueba y después en StoreInitial.
10. Registrar licencia y QA visual/técnica.

Regla de reimportación

No reimportar FBX para corregir una colocación de escena. Transform de importación, transform del prefab y transform de instancia son problemas distintos. La reimportación solo procede cuando cambia realmente el asset fuente o su configuración común.

27. Cómo crear prefabs y catálogos

Prefab

- raíz con escala 1,1,1 cuando sea posible
- pivot útil y orientación consistente
- visual separado de collider/interacción
- sin servicios globales
- referencias internas válidas
- variantes solo cuando comparten contrato
- LOD y materiales verificados

Catálogo

- ID estable y único
- referencia explícita al prefab/visual
- datos de gameplay separados del asset visual
- validación de duplicados y missing references

- orden determinista cuando afecte a tests
- fallback definido, no silencioso

Registro runtime

El runtime puede registrar assets aprobados mediante `RuntimeAssetRegistry`, pero no debe descubrir arquitectura recorriendo nombres de FBX. La escena y los catálogos declaran intención; el runtime valida y opera esa intención.

28. Cómo integrar personajes

1. Definir rol: cliente, dependiente, proveedor o placeholder técnico.
2. Separar perfil lógico de representación visual.
3. Validar escala, rig, animator, materiales y LOD.
4. Definir spawn, navegación, punto de interacción y fallback.
5. Evitar que una animación cambie el estado autoritativo sin comando explícito.
6. Probar oclusión, colisiones, cola, checkout y salida.
7. Comprobar diversidad visual sin convertirla en dependencia funcional.

La ropa de empleados debe conservar coherencia de marca VRM Games; clientes y proveedores pueden usar paletas y marcas variadas. Esa dirección visual no debe codificarse como regla de simulación.

29. Cómo integrar audio y VFX

Audio y VFX confirman estados, no los deciden. Deben dispararse a partir de eventos post-commit o resultados de interacción válidos.

Evento	Feedback mínimo	Riesgo
placement válido	sonido y visual de confirmación	disparar antes del commit
placement inválido	feedback no cromático + sonido	spam por frame
venta	sonido de checkout y resultado UI	duplicar por retry
save	confirmación discreta	afirmar éxito antes de escritura
apertura/cierre	ambiente y UI	desincronizar reloj lógico
error	señal clara y texto	usar solo color o sonido

Validación

- volumen relativo
- repetición y cooldown
- ausencia de null clips
- fallback silencioso registrado
- rendimiento de partículas
- accesibilidad sin depender solo de audio

30. Cómo revisar C#

La revisión busca preservar contratos, no solo estilo. Debe comprobar arquitectura, mutación, error, Unity lifecycle, rendimiento y testabilidad.

Área	Preguntas
Boundaries	¿el assembly depende solo de capas permitidas?
API	¿nombres, tipos y resultados expresan intención?
Null/error	¿el fallo es explícito y accionable?
Mutación	¿es atómica, idempotente y conserva invariantes?
Unity	¿Awake/OnEnable/Start/Update tienen responsabilidad limitada?
Referencias	¿se evita Find global en rutas críticas?
Colecciones	¿hay allocations o LINQ innecesario en hot paths?
Eventos	¿se suscribe y desuscribe correctamente?
Persistencia	¿snapshot y runtime no se mezclan?
Tests	¿cubren negativo, límite y regresión?

Cierre de revisión

No aprobar por “parece correcto”. Registrar comentarios bloqueantes y no bloqueantes, ejecutar pruebas pertinentes y revisar el diff final después de las correcciones.

31. Cómo ejecutar EditMode y PlayMode

1. Abrir el proyecto con la versión aprobada y esperar importación/compilación completa.
2. Confirmar ausencia de errores previos en Console.
3. Ejecutar primero la suite dirigida del sistema modificado.
4. Ejecutar EditMode completo cuando cambia lógica, datos, configuración o herramientas.
5. Ejecutar PlayMode completo cuando cambia escena, input, UI, composición o flujo.
6. Guardar reporte, versión, fecha, entorno y resultado.
7. Investigar flaky tests; no repetir hasta obtener verde sin causa.
8. Reiniciar Editor cuando se sospeche contaminación de estado y documentarlo.

Baseline de referencia

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La fotografía aceptada es 1215 EditMode y 70 PlayMode. Una cantidad distinta no es automáticamente fallo: puede existir un cambio aprobado de tests. Debe explicarse la diferencia y conservarse la trazabilidad.

32. Cómo ejecutar Golden Path

El Golden Path valida la cadena integrada que representa la promesa del vertical slice. No es una demostración improvisada; debe ejecutarse con precondiciones y resultado esperado registrados.

1. Arrancar desde Bootstrap o el punto definido por el gate.
2. Entrar en MainMenu y crear/cargar slot.
3. Acceder a la tienda representativa.

4. Preparar o recibir stock y colocarlo en displays.
5. Abrir la tienda y permitir llegada de clientes.
6. Completar browsing, reserva, carrito, cola y checkout.
7. Cerrar el día, revisar resultados y confirmar autosave.
8. Salir, volver a cargar y verificar continuidad.
9. Revisar Console o Player.log y registrar evidencia.

Fallo del Golden Path

Un fallo bloquea el gate si impide completar el flujo o corrompe estado. Un defecto cosmético puede registrarse con severidad menor, pero la decisión debe considerar la presentación exigida por H6.

33. Cómo realizar una campaña de siete días

La campaña comprueba acumulación, persistencia y economía más allá de un único recorrido. Debe usar un dataset reproducible o registrar las decisiones que afectan al resultado.

Día	Control principal
1	estado inicial, pedido, recepción y primera venta
2	persistencia de stock, caja y configuración
3	reservas, cola y repetición de checkout
4	gastos, margen y liquidez
5	save/load entre sesiones
6	carga, memoria y estabilidad acumulada
7	cierre, resultados agregados y snapshot final

Evidencia

- snapshot por día
- ledger y saldos
- inventario por contenedor
- número de clientes/ventas
- errores/logs
- tiempos de save/load
- defectos y decisiones de balance

34. Cómo hacer profiling

1. Definir hardware, resolución, calidad y build.
2. Definir escenario reproducible: tienda abierta, número de clientes y contenido.
3. Medir CPU, GPU, frame time, allocations, memoria y cargas.
4. Capturar baseline antes del cambio.
5. Identificar causa mediante profiler, no por intuición.

6. Aplicar una corrección por vez y comparar.
7. Ejecutar soak para degradación acumulativa.
8. Registrar objetivos y excepciones aprobadas.

No optimizar a ciegas

No eliminar claridad arquitectónica por microoptimización sin medición. Sí evitar búsquedas globales, allocations por frame, instanciación repetida y logs masivos en rutas críticas cuando los perfiles demuestren impacto o el patrón sea claramente riesgoso.

35. Cómo hacer balance

El balance se realiza después de tener un bucle estable. Los valores deben permanecer configurables y los cambios deben apoyarse en métricas de jornadas comparables.

1. Congelar dataset, seed y configuración de prueba.
2. Definir métricas: ventas, margen, stockouts, paciencia, cola, liquidez y duración.
3. Ejecutar varias jornadas sin cambiar parámetros.
4. Identificar cuello de botella o resultado dominante.
5. Ajustar el mínimo conjunto de valores.
6. Repetir y comparar distribuciones, no solo una partida.
7. Registrar baseline de balance y motivo.

Límites

- no introducir grind para alargar artificialmente
- no ocultar fallos de UX con números más fáciles
- no balancear una escena defectuosa
- no hardcodear excepciones de un único test

36. Cómo registrar defectos

Campo	Contenido
ID	estable
Resumen	síntoma observable
Build/versión	entorno exacto
Precondición	estado inicial
Pasos	reproducción mínima
Resultado actual	qué ocurre
Resultado esperado	contrato
Severidad	S0-S4
Sistema/requisito	trazabilidad
Evidencia	captura, log, save o video
Owner/estado	responsabilidad y ciclo de vida

Calidad del reporte

Un reporte útil permite a otra sesión reproducir el fallo sin conversación adicional. Evitar “no funciona” o “se ve mal” sin contexto. Para defectos visuales, incluir vista, escala y referencia aprobada.

37. Cómo aplicar severidades S0-S4

Severidad	Definición	Ejemplos
S0	pérdida grave, seguridad o imposibilidad total	corrupción irreversible, secreto expuesto
S1	bloqueo de flujo principal o invariante crítica	Golden Path imposible, duplicación de
S2	función importante degradada con workaround limitado	UI mueve jugador, visual duplicado, f
S3	impacto menor o deuda no bloqueante	alineación, warning, tooling pendiente
S4	cosmético o mejora	texto, pulido o preferencia sin impact

Contadores distintos

La QA Matrix cuenta defectos QA; producción y trazabilidad pueden incluir deuda con severidad de gate. Siempre indicar fuente y universo antes de decir “hay X S0/S1”.

38. Cómo decidir si un defecto bloquea

1. Comprobar severidad y reproducibilidad.
2. Identificar criterio o invariante afectados.
3. Comprobar si existe workaround compatible con el gate.
4. Evaluar riesgo de pérdida, duplicación, corrupción o falsa evidencia.
5. Evaluar frecuencia y alcance.
6. Decidir: bloquear, aceptar deuda temporal o diferir.
7. Registrar quién decide, por qué y hasta cuándo.

Regla H6

H6 exige cero S0/S1 dentro del universo de gate. Una reclasificación necesita evidencia y no puede usarse para bajar artificialmente la cifra. La deuda S2/S3 puede aceptarse solo con known issues y sin contradecir la experiencia representativa.

39. Cómo preparar una build

1. Confirmar working copy limpia o cambios intencionales identificados.
2. Fijar versión de aplicación, build ID y commit SHA.
3. Ejecutar preflight de Unity, paquetes, scenes/profile y configuración.
4. Confirmar Bootstrap, MainMenu y StoreInitial cuando corresponda; excluir TestLab de distribución.
5. Ejecutar suites requeridas.
6. Crear clean build Windows x64 con profile aprobado.
7. Arrancar EXE fuera del Editor.

8. Ejecutar smoke/Golden Path proporcional.
9. Revisar Player.log.
10. Empaquetar manifest, checksum, known issues y evidencia.

Tipos de build

Tipo	Uso
Development	iteración y diagnóstico
QA	campanas formales
H6 Candidate	gate del vertical slice
Demo/Alpha/Beta/RC	futuro; requiere apertura formal
Release/Hotfix	futuro; firma, distribución y rollback

40. Cómo revisar Player.log

1. Identificar log correspondiente a la ejecución, no uno anterior.
2. Buscar excepciones no controladas, errors, asserts y missing references.
3. Revisar warnings repetitivos y spam.
4. Correlacionar timestamps con pasos de prueba.
5. Distinguir error de aplicación, driver, entorno o cierre intencional.
6. Adjuntar extracto o archivo completo según política.
7. Crear defecto para fallos reproducibles y no silenciar el log.

Criterio

Una ventana que se abre no prueba una build. El proceso debe alcanzar el flujo esperado y el log no debe contener errores bloqueantes o recurrentes no aceptados.

41. Cómo archivar una build

Las builds de iteración pueden ser efímeras. Las candidatas de fase o hito deben conservarse con manifest y checksum.

Artefacto	Obligatorio en candidata
carpeta/ZIP de build	sí
Build ID y versión	sí
commit SHA	sí
Unity/package baseline	sí
scene list/profile	sí
resultados de tests	sí
Player.log	sí
known issues	sí
hardware/entorno	sí

Retención

Conservar al menos la candidata aprobada, la anterior utilizable para rollback y las builds necesarias para reproducir un defecto abierto. El resto puede purgarse según espacio, siempre que sus records permanezcan.

42. Cómo preparar una candidata de hito

1. Congelar alcance y known issues.
2. Cerrar o decidir formalmente defectos bloqueantes.
3. Repetir regresión completa y campañas requeridas.
4. Crear clean build desde SHA identificado.
5. Ejecutar fuera del Editor en entorno controlado.
6. Completar Golden Path y pruebas de persistencia.
7. Validar rendimiento, localización y accesibilidad del gate.
8. Archivar log, manifest, checksum y evidencia.
9. Convocar revisión de gate y registrar PASS/FAIL.
10. No modificar la candidata aprobada; cualquier cambio crea una nueva.

43. Cómo cerrar un sprint

Condiciones

- charter satisfecho
- work packages cerrados o decisión explícita
- tests y manual QA completados
- defectos triados
- build cuando la política la exige
- documentos y trazabilidad actualizados
- riesgos revisados
- handoff reproducible
- versión/SHA registrados

Procedimiento

1. Revisar cada criterio de aceptación y enlazar evidencia.
2. Comparar baseline de entrada y salida.
3. Registrar cambios, deuda aceptada y known issues.
4. Actualizar roadmap, Excel Maestro, QA Matrix y trazabilidad.
5. Crear closure record y, si aplica, tag/release interna.
6. Confirmar que el sprint siguiente cumple condición de apertura.

44. Cómo aprobar un hito

Un hito combina varios sprints y representa una capacidad de producto. La aprobación debe ser colegiada en términos documentales aunque el equipo sea una persona: producto, técnica, datos, UX, QA, build y gobierno se evalúan como perspectivas separadas.

Perspectiva	Pregunta
Producto	¿la promesa del hito es observable?
Técnica	¿la arquitectura soporta el resultado sin romper boundaries?
Datos	¿estado e invariantes son correctos y persistentes?
UX	¿el usuario entiende y completa el flujo?
QA	¿casos y defectos permiten PASS?
Build	¿funciona fuera del Editor?
Gobierno	¿cambios, evidencia y baseline son auditables?

45. Cómo actualizar producción

La fuente operativa es 12_Excel_Maestro_de_Produccion.xlsx. El documento 06 conserva intención de roadmap; el 12 mantiene estado vivo, work packages, consolidación, riesgos agregados y gates.

1. Actualizar estado de tarea y validación por separado.
2. Registrar dependencias, bloqueos y owner.
3. Actualizar riesgo si cambió probabilidad, impacto o mitigación.
4. Vincular defecto, build, versión y evidencia.
5. Ajustar capacidad relativa sin inventar horas consumidas.
6. Actualizar Sprint Activo y gates.
7. Revisar dashboard después de guardar.

No usar el Excel como

- sustituto de un requisito
- lugar para decisiones arquitectónicas completas
- registro de horas ficticias
- mecanismo para ocultar deuda
- fuente única de evidencia binaria

46. Cómo actualizar trazabilidad

La fuente append-only y de propagación es 13_Trazabilidad_y_Control_de_Cambios.xlsx. Debe registrar no solo cambios de código, sino regeneraciones, hashes, sustituciones, impactos, findings, excepciones y relaciones entre 00-34.

La cadena mínima es requisito → cambio → decisión → implementación → prueba → build → baseline. No todos los cambios necesitan ADR o build, pero las ausencias deben ser justificables.

1. Crear o actualizar Change ID.
2. Vincular requisitos y documentos impactados.

3. Vincular ADR cuando cambie arquitectura, datos, build o alcance.
4. Vincular código, escena, prefab o asset.
5. Vincular test, ejecución y defecto.
6. Vincular commit y build cuando existan.
7. Cerrar impacto documental y decisión final.
8. Ejecutar auditoría y revisar warnings/fails.

47. Cómo actualizar documentación

Toda actualización documental debe comenzar en 13, comprobar la autoridad en 14, aplicar el procedimiento de esta guía y terminar con impacto, revisión, hash externo y estado de baseline.

1. Identificar documentos competentes mediante el Binder.
2. Determinar si el cambio altera regla, estado, evidencia o historia.
3. Actualizar primero la fuente normativa más específica.
4. Actualizar documentos derivados y operativos afectados.
5. No copiar texto contradictorio sin revisar su autoridad.
6. Validar enlaces, IDs, tablas, versiones y terminología.
7. Calcular hash y registrar versión/cambio.
8. Marcar documento anterior como sustituido o histórico, no borrarlo.

Definition of Done documental

- sin contradicciones conocidas
- estado y fecha explícitos
- términos consistentes
- fuentes trazadas
- enlaces válidos
- historia preservada
- cambio y hash registrados

48. Cómo congelar una baseline

La congelación final se ejecuta con 32 como manifiesto, 16 como auditoría, los artefactos externos de integridad y una prueba de extracción/restauración. Un ZIP creado sin verificación posterior sigue siendo una copia, no una baseline aprobada.

1. Definir alcance de la baseline: documental, técnica, build o combinada.
2. Confirmar working copy, commit, versión de Unity y paquetes.
3. Ejecutar validaciones y gates exigidos.
4. Cerrar cambios y defectos del alcance o registrar deuda aceptada.
5. Exportar documentos y artefactos.

6. Calcular hashes y crear manifest.
7. Crear baseline record y handoff.
8. Proteger la baseline histórica de edición silenciosa.
9. Actualizar Binder y Guía Maestra si cambia la puerta de entrada.

Baseline no significa perfección

Una baseline puede contener known issues y deuda aceptada. Lo obligatorio es que su estado sea reproducible, explícito y no se presente como más completo de lo que es.

49. Cómo crear un handoff

Para una transferencia completa, el procedimiento detallado y los runbooks viven en `34_Final_Handoff_and_Project_Operations_Manual.md`; esta sección conserva la disciplina mínima aplicable a cualquier sesión o sprint.

Contenido mínimo

- fecha, rama, SHA y versión
- estado de sprint/hito
- qué está cerrado
- qué está en working copy
- tests y build aceptados
- defectos y riesgos abiertos
- tarea siguiente exacta
- archivos que deben leerse
- comandos o pasos de reproducción
- acciones prohibidas o precauciones

Criterio de calidad

Una nueva sesión debe poder continuar sin consultar chats previos. El handoff no debe ocultar diferencias entre commit publicado y working copy local.

50. Cómo recuperar una máquina o proyecto

1. Instalar Unity Hub y Unity 6000.3.18f1 con módulos aprobados.
2. Clonar o restaurar repositorio sin copiar Library, Temp, Logs u Obj.
3. Confirmar Packages/manifest.json, lock y ProjectSettings.
4. Abrir con la versión correcta y permitir importación.
5. Revisar Console y referencias perdidas.
6. Ejecutar smoke/EditMode/PlayMode según guía de setup.
7. Validar escenas y Build Profile.
8. Crear build mínima y revisar Player.log.

9. Restaurar evidencia o caches solo si están documentados, nunca como sustituto del source.

Limpieza segura

Cerrar Unity antes de eliminar Library. No borrar .meta, ProjectSettings, Packages ni assets para “forzar” una solución. Una regeneración debe seguirse de compilación y tests.

51. Cómo actualizar Unity o paquetes

1. Justificar necesidad, beneficio y riesgo.
2. Crear ADR y baseline de entrada.
3. Leer cambios del proveedor y compatibilidad de URP/Input/Navigation/Tests.
4. Probar en rama o working copy aislada.
5. Actualizar un componente lógico cada vez.
6. Hacer clean import y revisar diffs de ProjectSettings/lock.
7. Ejecutar suites, escenas, Golden Path y build.
8. Comparar rendimiento, warnings y serialización.
9. Decidir aceptar, aplazar o revertir.
10. Actualizar setup, build guide y baseline.

Prohibición

No actualizar paquetes “porque hay una versión nueva” durante Sprint 16/17 sin necesidad bloqueante. La estabilidad del gate prevalece sobre novedad.

52. Cómo gestionar secretos, seguridad y licencias

Aplicar conjuntamente 23, 26, 27 y 33. Legal autoriza uso/distribución; privacidad limita datos; seguridad protege activos y responde a incidentes; continuidad define recuperación y evidencia.

Secretos

- no incluir tokens, claves, contraseñas o credenciales en repo
- usar configuración externa futura para servicios
- revisar logs, manifests y screenshots
- rotar un secreto expuesto y registrar incidente

Licencias

- registrar origen y licencia de cada asset externo
- conservar texto de licencia y atribución
- distinguir concept art interno de asset distribuible
- no asumir que una imagen de referencia autoriza uso comercial
- revisar dependencias antes de demo o release

Archivos de usuario

Los saves deben tratarse como datos locales del jugador, con escritura segura y sin incluir información innecesaria. Los logs no deben exponer rutas o datos sensibles más allá de lo necesario para diagnóstico.

53. Cómo controlar scope creep

1. Comparar la propuesta con 00, 02 y el sprint charter.
2. Clasificarla como necesaria para gate, mejora posterior, visión o exclusión.
3. Estimar impacto en sistemas, datos, UX, QA, build y documentación.
4. Comprobar si desplaza una tarea P0 o aumenta WIP.
5. Rechazarla para el sprint actual si no es necesaria para el resultado.
6. Registrarla en roadmap futuro sin fecha si merece conservarse.
7. Crear cambio de alcance y ADR si se acepta formalmente.

Pregunta decisiva

“¿H6 dejaría de ser válido sin esta feature?” Si la respuesta es no, probablemente no pertenece a Sprint 16/17. Puede ser valiosa y aun así estar diferida.

54. Cómo tratar sistemas futuros

Empleados, investigación, puestos informáticos, comercio online, publishing, desarrollo interno, plataforma e infraestructura están documentados como evolución posible. No forman parte del compromiso inmediato.

Bloque	Estado	Condición de apertura
Empleados	cercano post-H6	baseline y revisión PVS-0
Investigación	MVP futuro	empleados/roadmap aprobado
Puestos informáticos	MVP futuro	placement y servicio diseñados
Online/logística	expansión	H7 y arquitectura de inventario reutilizada
Publishing	visión	H8 y negocio multicanal
Desarrollo interno	visión	publishing/conocimiento comercial
Plataforma/infraestructura	visión tardía	H9/H10 y capacidad real

Regla de diseño futuro

Documentar interfaces y límites puede ser útil; implementar stubs, tablas, menús o datos que no participan en el gate actual añade mantenimiento y debe evitarse.

54.1. Cierre formal de Sprint 16

Sprint 16 fue cerrado como COMPLETED / PASS el 2026-07-06 sobre la build 0.0.26. La aprobación incluye StoreInitial, integración funcional, input UI/mundo, suites automatizadas, regresión manual, Golden Path externo, persistencia tras reinicio y revisión de Player.log. No quedan S0/S1 conocidos dentro del alcance de cierre.

55. Cómo abrir Sprint 17

Sprint 17 es estabilización, no continuación informal de Sprint 16. Solo se abre cuando la presentación representativa está integrada y existe una baseline post-integración.

- Sprint 16 CLOSED / PASS
- StoreInitial autorada y conectada
- shell procedural desactivado sin duplicación
- input UI/mundo corregido
- suites completas en PASS
- Golden Path post-integración
- build Windows x64 externa y Player.log
- defectos S0/S1 de S16 cerrados o decisión compatible con gate
- documentación y trazabilidad actualizadas

Workstreams

Workstream	Resultado
Balance	dataset, métricas y baseline
Rendimiento	profile, soak y correcciones
Persistencia	slots, fault injection y equivalencia
UX/Accesibilidad	HUD, foco, ES/EN y resoluciones
Defectos	triage y cero S0/S1
Regresión	suites, Golden Path y siete días
Build	QA y candidata H6
Documentación	evidencia y baseline

56. Cómo aprobar H6

La decisión formal debe ejecutarse en el documento 31 contra el contrato 02. Esta guía describe el procedimiento, pero no puede convertir controles NOT RUN en PASS ni reemplazar firmas y evidencias.

H6 es la aprobación del vertical slice, no el cierre administrativo de una lista. Debe demostrar una tienda representativa, jugable, persistente, estable, medible y auditable.

1. Confirmar escena y contenido representativos aprobados.
2. Completar Golden Path en build externa.
3. Completar campaña persistente de siete días.
4. Ejecutar regresión completa sin fallo bloqueante.
5. Confirmar cero S0/S1 dentro del universo de gate.
6. Revisar Player.log y known issues.
7. Validar rendimiento y soak en hardware registrado.
8. Validar ES/EN, resoluciones y accesibilidad básica.
9. Crear candidata identificada, empaquetada y con checksum.

10. Cerrar documentación, trazabilidad y decisión PASS/FAIL.

PASS with Accepted Debt

Solo puede usarse para deuda no bloqueante, explícita, con mitigación y trigger. No aplica a pérdida de datos, duplicación, imposibilidad de Golden Path, corrupción, fallo grave de build o S0/S1.

57. Cómo preparar la fase post-H6

La fase posterior se apoya en 24, 25, 28 y 31 y requiere una decisión separada. H6 aprobado no implica automáticamente abrir Steam, publicar página, distribuir demo o anunciar fecha.

1. Congelar baseline H6 y no continuar sobre una working copy ambigua.
2. Realizar retrospectiva de producto, técnica, QA y documentación.
3. Revisar métricas de uso, balance, rendimiento y deuda.
4. Reclasificar backlog futuro por valor, coste y riesgo.
5. Decidir si el siguiente objetivo es profundizar tienda o preparar una demo.
6. Abrir solo un bloque: empleados, investigación o puestos según roadmap aprobado.
7. Crear nuevo charter, ADRs y QA antes de implementar.
8. Mantener online, publishing y plataforma cerrados salvo decisión formal.

58. Checklists operativos

Checklist diaria

- rama/SHA revisados
- una tarea principal
- criterio y evidencia definidos
- bloqueos visibles
- tests dirigidos ejecutados
- diff revisado
- estado y handoff actualizados

Checklist feature

- requisito y ADR
- invariantes
- datos configurables
- tests negativos
- persistencia
- UI/input
- logs
- documentos y trazabilidad

Checklist escena

- GUID/meta
- roots
- referencias
- colliders
- navigation
- lighting
- prefabs
- sin managers duplicados
- PlayMode
- build

Checklist build

- versión
- SHA
- profile
- escenas
- tests
- clean build
- EXE externo
- Player.log
- manifest
- checksum
- known issues

Checklist cierre

- criterios
- evidencia
- defectos
- riesgos
- producción
- QA Matrix
- trazabilidad
- build
- closure record
- handoff

59. Plantillas reutilizables

Plantilla de tarea

ID:

Título:

Fuente / requisito:

Sprint / gate:

Objetivo:

Precondiciones:

Dependencias:

Pasos previstos:

Criterio de aceptación:

Evidencia requerida:

Riesgos:

Estado implementación:

Estado validación:

Build / SHA:

Notas de cierre:

Plantilla de defecto

Defect ID:

Versión / Build / SHA:

Severidad:

Precondición:

Pasos:

Resultado actual:

Resultado esperado:

Frecuencia:

Evidencia:

Requisito / sistema:

Workaround:

Decisión de gate:

Plantilla de revisión de gate

Gate:

Baseline de entrada:

Criterios evaluados:

Evidencia:

Defectos abiertos:

Deuda aceptada:

Build candidata:

Logs y métricas:

Documentos actualizados:

Decisión: PASS / FAIL / PASS WITH ACCEPTED DEBT

Responsable y fecha:

Plantilla de handoff

Fecha / rama / SHA / versión:

Estado global:

Último resultado validado:

Cambios locales:

Tests y build:

Defectos / riesgos:

Siguiente tarea exacta:

Archivos que leer:

Acciones prohibidas:

Comandos o pasos de reproducción:

60. Glosario, problemas frecuentes y mantenimiento de la guía

Glosario

Término	Definición
Baseline	estado identificado y reproducible usado como referencia
Gate	condición de entrada/salida con decisión PASS/FAIL
Golden Path	flujo integrado principal del vertical slice
ADR	registro de decisión arquitectónica
WIP	trabajo iniciado pero no cerrado
DoD	condiciones completas de cierre
Snapshot	representación serializable del estado
Schema	versión y estructura del contrato persistido
Preflight	validaciones previas a una mutación o build
Idempotencia	repetir una operación no duplica su efecto
Fallback	alternativa controlada ante ausencia/fallo
Working copy	estado local todavía no necesariamente publicado
H6	hito de aprobación del vertical slice

Problemas frecuentes

Problema	Diagnóstico	Respuesta
Tests verdes pero sprint abierto	falta aceptación visual/build/evidencia	completar gate, no cerrar por a
Store y StoreInitial divergen	migración parcial	mantener fallback, registrar su
Clic UI mueve jugador	input compartido	prioridad EventSystem y test d
Save primario corrupto	fallo de archivo	backup-first y restore en dos fa
Build difiere del Editor	scene/profile/config	clean build, log y preflight
Dashboard parece incoherente	universos distintos	abrir fuente y nombrar contado
Asset mal colocado	transform de instancia	no reimportar FBX sin cambio t
Cambio toca muchos documentos	impacto transversal	crear Change ID/ADR y matriz

Mantenimiento

Esta guía debe revisarse cuando cambia el proceso, no por cada feature. Un cambio de jerarquía documental, política de gate, baseline de Unity, estrategia de build, schema, ownership de escena o workflow de producción puede requerir actualización. Las instrucciones específicas de un sistema deben permanecer en su documento especializado para evitar que esta guía se convierta en una copia desactualizada.

Criterios de aceptación de la Guía Maestra

- Permite iniciar y cerrar una sesión sin contexto de chat.
- Distingue fuente normativa, estado operativo y procedimiento.
- Cubre producto, ingeniería, escenas, contenido, QA, build y gobierno.
- Refleja Sprints 0-17, H6 y Vertical Slice cerrados y validados.
- Mantiene StoreInitial como trabajo inmediato y sistemas futuros cerrados.

- Explica los distintos universos de defectos/deuda.
- Incluye checklists y plantillas reutilizables.
- No contradice la jerarquía del Binder.

Playbooks end-to-end de referencia

Los siguientes playbooks muestran cómo combinar los capítulos anteriores en situaciones reales. No sustituyen al requisito específico, pero evitan que cada incidencia se resuelva con un proceso improvisado. Cada playbook termina con una condición de cierre y una lista de evidencias.

Playbook A — Corregir una desviación visual de StoreInitial Una desviación visual puede parecer menor, pero en Sprint 16 puede bloquear la aprobación representativa. El primer paso es identificar si el problema pertenece al asset fuente, al prefab, a la instancia de escena o a la lógica runtime. Por ejemplo, un muro desplazado suele ser un problema de autoría de escena; una escala incorrecta compartida por todas las instancias puede ser importación; una puerta que cambia de posición al entrar en Play puede revelar que el runtime compite por su transform.

Se debe congelar una vista y referencia aprobada, registrar la escena y commit, y reproducir el problema sin herramientas automáticas que alteren jerarquía. Después se revisan pivotes, parent, escala local, constraints, colliders y scripts que escriben transform. No debe reimportarse el FBX por defecto ni compensar con offsets ocultos en código. La solución preferida es que la escena autorada contenga la posición correcta y que StoreInitialSceneContext proporcione referencias explícitas a los componentes que el runtime necesita operar.

Tras corregir, se valida en Edit Mode, Play Mode y recorrido manual. Hay que comprobar cámara, navegación, acceso, oclusión, puerta, zonas y relación con mobiliario. Si el objeto forma parte de un prefab, se decide si la corrección pertenece a la instancia o debe aplicarse al prefab sin perjudicar otros usos. La evidencia incluye captura antes/después, diff de escena/prefab, validación de referencias, test dirigido y registro del defecto. El playbook se cierra cuando la desviación deja de reproducirse, no aparecen duplicados al entrar en Play y la corrección no altera placement, navegación o build.

Playbook B — Corregir propagación de input entre UI y mundo Se reproduce el fallo con un recorrido mínimo: abrir el panel afectado, colocar el puntero sobre el control, hacer clic y observar si el jugador se mueve, coloca un objeto o interactúa con el mundo. El reporte debe indicar tipo de control, posición del puntero, action map activo, escena, resolución y si existe modal. Se revisa el orden de procesamiento entre EventSystem, router de input, control de contexto y consumidores de mundo.

La corrección debe establecer una única decisión de consumo. No es suficiente añadir comprobaciones distintas en cada feature, porque aparecerían divergencias entre movimiento, placement y otras interacciones. El punto común debe saber si el puntero está sobre UI, si una vista modal posee foco o si el contexto gameplay está suspendido. También debe conservar navegación válida cuando la UI se cierra y evitar que el clic de cierre se reutilice por el mundo en el mismo frame.

Las pruebas cubren Button, Toggle, Scroll, área vacía del panel, overlay transparente y clic fuera de UI. Debe probarse que el mundo responde en una zona libre y que teclado/ratón no quedan bloqueados tras cerrar el panel. Si el proyecto usa callbacks y polling, se comprueba que no existan dos rutas activas. La evidencia mínima es test PlayMode o integración, ejecución manual, ausencia de movimiento no solicitado y registro del cambio. El defecto se cierra únicamente cuando todos los consumidores de mundo respetan la misma política y no existe un workaround específico de una ventana.

Playbook C — Introducir una migración de schema Antes de modificar un snapshot se identifica qué necesidad de producto o integridad obliga al cambio. Añadir un campo opcional no siempre requiere incremento, pero cambiar significado, unidad, estructura o invariantes sí puede hacerlo. Se documentan schema origen, destino, compatibilidad y comportamiento ante versiones

futuras. La migración debe transformar datos sin mutar el runtime hasta que el resultado completo sea válido.

Se preparan fixtures de la versión anterior, actual, corrupta y futura. La migración se ejecuta sobre una copia o representación intermedia; se validan IDs, cantidades, dinero en unidades menores, relaciones y ausencia de duplicados. El backup válido no debe ser destruido por una migración fallida. Si el proyecto no puede migrar una versión, debe rechazarla con error explícito y conservar los archivos.

Se actualizan Modelo de Datos, TDD, QA Plan/Matrix, Build Guide y trazabilidad. La build que contiene la migración debe identificarse y ejecutar round trip, restore de backup, interrupción de escritura y equivalencia de estado. La evidencia incluye archivos de entrada/salida, hashes, resultados y decisión de rollback. El cambio se cierra cuando una versión anterior soportada carga de manera determinista, un archivo futuro se rechaza sin daño y un fallo intermedio no altera el estado activo ni el único backup.

Playbook D — Investigar una pérdida o duplicación de inventario Se comienza por reconstruir el movimiento exacto de unidades: contenedor origen, destino, producto, cantidad, reserva, operación y momento del fallo. Debe comprobarse si el síntoma procede de la UI, una proyección desactualizada o el estado autoritativo. El principio de conservación permite comparar suma total antes y después, incluyendo stock reservado, cajas, displays, carrito y unidades comprometidas en checkout.

La investigación revisa preflight y commit. Cualquier mutación previa a validar capacidad, disponibilidad o destino puede dejar estado parcial. También se inspeccionan retries, eventos duplicados y cargas de save que vuelvan a aplicar una operación. La corrección debe centralizar la transferencia y devolver un resultado explícito; no debe ajustar cantidades después del fallo para “cuadrar” el total.

Las pruebas cubren cantidad cero/negativa, insuficiencia, capacidad exacta, destino lleno, mismo origen/destino, cancelación, repetición y persistencia. Se añade un test de invariante que calcule la suma total y un test de regresión para la secuencia observada. Si existe daño de save, se documenta recuperación o incompatibilidad. El cierre exige conservación de unidades, ausencia de cantidades negativas, idempotencia donde corresponda y equivalencia tras save/load.

Playbook E — Preparar y decidir una candidata H6 La preparación comienza cerrando el alcance: no se aceptan features nuevas, actualizaciones de paquetes ni cambios visuales no justificados después del freeze. Se selecciona un commit candidato, se confirma working copy limpia y se ejecutan preflight, suites, Golden Path y campaña persistente. Los defectos se triagean contra criterios H6 y se separa el contador QA del contador combinado de producción/deuda.

La build se genera limpia con escenas aprobadas y sin TestLab. El ejecutable se prueba fuera del Editor, con hardware, resolución y calidad registrados. Se revisan Player.log, tiempos de carga, save/load, memoria, frame time y soak. Se comprueban ES/EN, truncamientos, foco, feedback no cromático y recuperación de archivos. Todo resultado debe asociarse a Build ID y SHA.

El paquete de revisión incluye manifest, checksum, test reports, defectos abiertos, deuda aceptada, known issues, capturas y decisión recomendada. PASS requiere que no existan S0/S1 en el universo de gate y que la deuda restante no contradiga la promesa representativa. FAIL debe indicar qué criterio falla y qué trabajo reabre. La candidata no se parchea; cualquier corrección produce un nuevo ID y repite validación proporcional.

Playbook F — Actualizar un paquete de Unity Una actualización de paquete comienza con una necesidad: defecto bloqueante, soporte requerido o riesgo de seguridad. Se registra la versión actual, objetivo, dependencias transitivas y documentos afectados. Durante Sprint 16/17 una actualización por conveniencia debe rechazarse. El experimento se realiza en una rama o copia aislada con baseline de tests y build.

Se cambia un paquete lógico cada vez, se revisan manifest.json y lock, y se hace clean import cuando sea necesario. Se inspeccionan errores de compilación, serialización de assets, shaders,

Input System, Navigation y Test Framework. Los cambios automáticos de ProjectSettings se revisan línea por línea. No se aceptan paquetes preview salvo decisión explícita.

La validación incluye suites completas, apertura de escenas, Golden Path, build externa, Player.log y comparación de rendimiento. Si la actualización no aporta beneficio neto o introduce deuda no asumible, se revierte por completo. Si se acepta, se actualizan ADR, Setup Guide, Build Guide, manifest de baseline y registros de licencia. El cierre exige reproducibilidad en una working copy limpia.

Playbook G — Incorporar un asset representativo Primero se confirma que el asset pertenece al set representativo aprobado y que su licencia permite el uso previsto. Se conserva archivo fuente y versión. La importación valida unidad, orientación, pivote, jerarquía, materiales, texturas, normals y LOD. Después se crea un prefab de proyecto; el FBX no se usa como composición final si necesita collider, interacción o referencias adicionales.

El prefab separa visual, collider, puntos de interacción y metadatos. Se asigna ID estable y catálogo si el runtime debe resolverlo. El asset se prueba de forma aislada y en StoreInitial con iluminación, cámara, navegación y escala relativa. Para mobiliario se comprueba huella y acceso; para productos, legibilidad; para personajes, rig y animación; para arquitectura, continuidad y oclusión.

La QA registra defectos técnicos y visuales por separado. La aceptación no se basa únicamente en que el objeto aparezca: debe integrarse sin missing materials, escalas compensadas, colliders imposibles o dependencia de nombres importados. El cierre incluye licencia, prefab, catálogo, validación, capturas y trazabilidad.

Playbook H — Resolver una regresión de rendimiento Se reproduce en la misma build, hardware, resolución y escenario que la baseline. Se evita comparar Editor con Player o escenas con distinta población. Se captura frame time CPU/GPU, memoria, allocations y spikes. La regresión se acota mediante comparación de commits o desactivación controlada de subsistemas, sin asumir que el asset más reciente es culpable.

Una vez identificado el hot path, se analiza causa: búsqueda por frame, material/renderer excesivo, sombras, animación, navegación, UI rebuild, logging o serialización. La solución debe medirse. Puede consistir en cache, pooling, LOD, batching, reducción de frecuencia o cambio de algoritmo; no debe eliminar feedback o validación sin evaluar producto.

Se repite el escenario y soak, se registran métricas antes/después y se comprueba no regresión funcional. Si el objetivo no está definido, se crea una decisión de performance antes de declarar PASS. El cierre exige mejora reproducible o deuda aceptada con límite y trigger.

Playbook I — Validar localización y accesibilidad Se prepara una matriz de idioma, resolución, escala UI y flujo. ES y EN deben usar tablas o recursos, no cadenas dispersas. Se recorren MainMenu, slots, HUD, Operations, tutorial, errores, resultados y modales. Se buscan claves crudas, truncamientos, solapamientos, saltos de línea incorrectos y texto que dependa de género o plural no contemplado.

La accesibilidad básica comprueba legibilidad, contraste, alternativas a color, feedback visual y sonoro, foco, tamaños de objetivo y consistencia de controles. La prueba no consiste en activar una opción aislada; debe verificar que el Golden Path sigue siendo comprensible. Los mensajes de error deben indicar acción siguiente.

Los defectos se vinculan a pantalla, idioma, resolución y build. Las correcciones se prueban en ambos idiomas para evitar que un layout específico rompa el otro. El cierre H6 exige que no existan bloqueos de flujo ni claves visibles y que las señales críticas no dependan exclusivamente del color.

Playbook J — Evaluar una solicitud de alcance La solicitud se describe sin convertirla inmediatamente en tarea. Se compara con la fantasía, el contrato H6 y el sprint activo. Se identifica beneficio, coste, dependencias, datos, UI, QA, build y riesgo. Una idea puede encajar con la visión y no pertenecer al alcance actual.

Se aplican cuatro resultados: aceptar ahora porque es necesaria para el gate; diferir a post-H6; mantener como visión sin compromiso; rechazar porque contradice pilares o coste. Aceptar requiere Change ID, impacto documental, roadmap, capacidad y posiblemente ADR. Diferir requiere condición de reapertura, no una fecha inventada.

La decisión debe proteger el trabajo ya cerrado. Por ejemplo, empleados o investigación no deben añadirse a Sprint 17 porque transformarían estabilización en desarrollo de sistemas. El cierre es una decisión registrada y comunicada, no una fila escondida en backlog.

Playbook K — Recuperar el proyecto tras una corrupción local Se detiene Unity y se preservan logs y archivos afectados antes de limpiar. Se determina si el problema está en Library/cache, un asset versionado, .meta, ProjectSettings o repositorio. No se borran archivos versionados para probar al azar. Se compara con Git y con la baseline hashada.

Si el problema es cache, se elimina únicamente lo regenerable con Unity cerrado. Si existe GUID perdido o conflicto de meta, se restaura el par asset/meta correcto. Si una escena o prefab está dañado, se recupera desde commit y se reaplican cambios controlados. Después se abre con la versión exacta, se espera importación, se revisan errores y se ejecutan smoke tests.

La recuperación termina con suites, escenas y build proporcional. Se documenta causa para evitar repetición. Si el problema revela que una salida esencial no estaba versionada, se corrige la política de setup y baseline.

Playbook L — Resolver una contradicción documental Se recopilan los textos conflictivos y se etiqueta cada uno como intención, implementación, evidencia, historia o visión. Se consulta la jerarquía, especificidad, fecha y estado. Después se contrasta con código, ProjectSettings, test o build si la contradicción afecta al estado real.

La resolución puede requerir corregir un documento, corregir la implementación, aceptar un cambio o mantener ambas afirmaciones con contexto temporal. Nunca se elige el texto que resulta más conveniente sin registrar decisión. Se crea Change ID, impacto documental y ADR cuando afecta arquitectura o alcance.

El cierre actualiza todas las fuentes derivadas, añade sustitución si corresponde y recalcula hashes. El Binder y la Guía se actualizan solo si cambia navegación o procedimiento. La contradicción se considera resuelta cuando un lector puede determinar qué era cierto antes, qué es cierto ahora y por qué cambió.

Definition of Ready por tipo de trabajo

Tipo	Debe estar listo antes de empezar
Feature	requisito, criterio, sistema, datos, dependencia, test y sprint autorizado
Bug	reproducción, build/versión, severidad, esperado y evidencia
Refactor	problema medido, contratos preservados, tests de caracterización y límite
ADR	contexto, alternativas, restricciones y decisión necesaria
Schema	origen/destino, compatibilidad, fixtures y rollback
Escena	layout, referencias, roots, responsables y criterio visual
Asset	origen/licencia, carpeta, escala, uso y aceptación
UI	flujo, estados, input, copy y resolución
QA	caso, precondition, build, datos y resultado esperado
Build	SHA, versión, profile, escenas, tests y artefactos
Documento	autoridad, fuentes, impacto y estado objetivo
Cambio de alcance	beneficio, coste, gate, alternativas y capacidad

Definition of Ready no pretende bloquear toda exploración. Un spike puede empezar con incertidumbre explícita, tiempo limitado y una pregunta de salida. Lo que no debe ocurrir es iniciar implementación productiva sin saber cómo se aceptará o qué documento posee la decisión.

Definition of Done por tipo de trabajo

Tipo	Condición de cierre
Feature	contrato implementado, tests, integración, persistencia, evidencia y documentos
Bug	causa corregida, regresión, evidencia, triage y versión/build
Refactor	comportamiento preservado, deuda reducida, métricas/tests y diff revisado
ADR	decisión aceptada, consecuencias e impacto registrados
Schema	migración, compatibilidad, fallos seguros, build y fixtures
Escena	autoría, referencias, navegación, visual, PlayMode y build
Asset	licencia, import, prefab, catálogo, QA y trazabilidad
UI	flujo, foco, input exclusivo, idiomas, resoluciones y accesibilidad
QA	ejecución reproducible, resultado, evidencia y defectos
Build	EXE externo, log, manifest, checksum, known issues y archivo
Documento	coherencia, versión, hash, cambio, revisión y sustitución
Sprint	charter, work packages, QA, build, documentación, closure y handoff

Taxonomía de evidencia

Evidencia	Demuestra	No demuestra por sí sola
Test EditMode	reglas puras, datos, configuración o herramientas	integración visual completa
Test PlayMode	comportamiento Unity automatizado	calidad visual o experiencia humana
Captura	estado visual puntual	flujo, interacción o ausencia de errores
Video	secuencia observable	integridad interna o reproducibilidad
Player.log	errores y eventos de ejecución	que todo requisito se haya recorrido
Build/EXE	reproducibilidad fuera del Editor	que el Golden Path esté aprobado
Save fixture	estructura y datos persistidos	que la restauración runtime sea correcta
Profiler capture	coste de un escenario	rendimiento en todos los hardware/
Checksum	integridad del artefacto	calidad o corrección funcional
Commit SHA	identidad del source	que la build proceda realmente de él
Closure record	decisión y contexto	evidencia técnica si no está enlazado
Aprobación visual	adecuación representativa	invariantes, save o build

La evidencia debe ser proporcional al riesgo. Una corrección de texto no requiere campaña de siete días; un cambio de save no puede cerrarse con una captura. Combinar evidencias evita que una única señal se use fuera de su alcance.

Cadencia de mantenimiento operativo

Cada sesión

- revisar working copy
- elegir tarea
- actualizar estado si cambia
- ejecutar pruebas dirigidas
- dejar handoff breve

Cada bloque de integración

- regresión del sistema
- revisión de dependencias
- actualizar riesgos
- validar escena/build cuando aplique
- cerrar impactos documentales

Cada semana o revisión equivalente

- dashboard y WIP
- riesgos High/Critical
- defectos/deuda
- capacidad
- builds y gates
- documentos/ADRs abiertos

Cada sprint

- baseline entrada/salida
- charter y cierre
- tests completos
- build de política
- closure record
- handoff
- actualización Binder si cambia navegación

Cada hito

- freeze
- candidata
- auditoría end-to-end
- retrospectiva
- baseline hashada
- decisión de siguiente fase

Heurísticas de decisión y señales de parada

Señal	Detenerse cuando	Siguiente acción
Incertidumbre arquitectónica	hay dos ownership plausibles	spike/ADR antes de código
Tests fallan en áreas no relacionadas	la working copy mezcla cambios	aislar/revertir y reducir WIP
Escena se corrige con offsets crecientes	runtime y autoría compiten	definir referencias y owners
Save requiere parches por versión	no existe estrategia de migración	diseñar schema y fixtures

Señal	Detenerse cuando	Siguiente acción
Build solo funciona en Editor	profile/scene/config diverge	preflight y build limpia
Dashboard mejora al reclasificar	se está maquillando el gate	volver a evidencia y definición
Feature futura parece “pequeña”	abre nuevas entidades/UI/save	control de alcance
Asset necesita scripts por nombre	importación dirige arquitectura	crear prefab/contexto explícito
Corrección obliga a cambiar test esperado	puede ocultar regresión	revisar contrato/ADR
No se puede explicar el diff	sesión demasiado amplia	dividir o restaurar baseline

Matriz de impacto documental por cambio

Cambio	Revisión mínima
Regla de gameplay	00 si afecta dirección; 01; 02; 05; 06; 07/08; 12/13
Arquitectura	03; 09; ADR; tests; 12/13; 10/11 si afecta entorno/build
Entidad o invariante	04; 03; 07/08; save/migraciones; 13
Flujo UX/input	05; 03; 07/08; 12/13; 02 si afecta H6
Escena/StoreInitial	02; 03; 05; 06; 07/08; 10; 11; 12/13
Unity/paquete	10; 11; ADR; tests/build; 13; Binder si cambia baseline
Build/release	11; 07/08; 06; 12/13; baseline/handoff
Alcance	00; 01; 02; 06; 12/13; 14/15 si cambia navegación/proceso
Documento consolidado	14; 13; manifest/hash; 15 solo si cambia procedimiento
Asset/licencia	10; inventario de contenido; 12/13; build/legal futura

Protocolo de revisión final de una sesión

1. Comparar el objetivo inicial con el resultado real; registrar desviaciones.
2. Revisar git diff incluyendo .meta, ProjectSettings y archivos binarios añadidos.
3. Eliminar archivos temporales o generados que no deben versionarse.
4. Ejecutar la prueba dirigida final después del último cambio, no confiar en una ejecución anterior.
5. Comprobar Console o logs para errores que no afectaron visiblemente al flujo.
6. Actualizar implementación y validación como estados independientes.
7. Vincular evidencia, defecto, cambio y build cuando corresponda.
8. Dejar la working copy en un estado comprensible: cerrado, intencionalmente parcial o revertido.
9. Escribir la siguiente acción exacta y su precondition.
10. No hacer push o tag hasta revisar alcance del commit y secretos/licencias.

Manual operativo por sistema del vertical slice

Esta sección resume cómo intervenir en cada sistema sin perder la relación entre regla, estado, persistencia y evidencia. Cada mapa debe leerse junto al GDD, TDD, Modelo de Datos y QA Matrix.

Aplicación, Bootstrap y sesión Bootstrap debe crear o localizar una única raíz de aplicación, preparar servicios persistentes y conducir al menú sin depender de que una escena gameplay haya sido abierta directamente. Cualquier cambio debe probar arranque frío, retorno a menú, salida y

recarga de dominio. El menú no debe contener servicios globales duplicados ni crear una sesión más de una vez.

Las operaciones New Game, Continue, Replace y Delete deben actuar sobre slots identificados y presentar confirmaciones para acciones destructivas. La sesión seleccionada debe propagarse de forma explícita al flujo de carga. Los tests deben detectar inicialización duplicada, raíces persistentes múltiples, navegación a una escena inexistente y pérdida de guardado al volver al menú. La evidencia adecuada combina PlayMode, recorrido desde Bootstrap y build externa.

Movimiento, cámara y navegación El movimiento click-to-move convierte una intención de mundo en un destino válido, rechaza puntos inaccesibles y nunca se activa por input consumido por UI. La cámara orbital y el zoom deben respetar límites configurados, mantener legibles entrada, displays, cola y checkout, y evitar penetrar arquitectura de forma que impida el flujo.

Una modificación de navegación se prueba en puntos obligatorios, esquinas, entrada, almacén y áreas cerca de mobiliario. Deben existir rutas negativas para destinos bloqueados. Si cambia StoreInitial, se comprueba que colliders visuales y superficie navegable no se contradigan. El estado de movimiento no debe persistir como una operación económica; sí puede ser necesario restaurar posición y contexto de sesión según el snapshot vigente.

Grid, placement y retirada El grid de la tienda usa celdas de 0,5 m y una representación lógica independiente de la escala visual. La previsualización debe coincidir con huella, rotación y posición final. Antes del commit se validan límites, ocupación, reservas de acceso y conectividad. La ocupación debe aplicarse atómicamente: un fallo no puede dejar celdas parciales.

La retirada libera exactamente la huella ocupada y devuelve o elimina el objeto según la regla de producto. La arquitectura fija de StoreInitial no se trata como placement del jugador. El mobiliario inicial y dinámico debe distinguirse mediante roots, datos o procedencia. Las pruebas cubren bordes, rotaciones, solapamientos, acceso a entrada, retirada, repetición, save/load y compatibilidad con objetos ya persistidos.

Productos, definiciones y catálogos Una definición de producto posee ID estable, nombre/localización, categoría, coste, precio base o parámetros configurables y representación visual. El runtime no debe usar el nombre visible como identidad. Un catálogo valida IDs duplicados, referencias ausentes y consistencia entre datos y prefabs.

Al añadir un producto se actualizan datos, visual, catálogo, inventario inicial o proveedor cuando corresponda, pruebas y localización. No debe aparecer automáticamente en todos los sistemas por búsqueda de assets. La evidencia incluye validación de catálogo, carga del runtime registry, aparición en pedidos/displays y persistencia de cantidades con el mismo ID.

Inventarios y contenedores Cada contenedor declara capacidad, reglas de aceptación y cantidades no negativas. El servicio de transferencia es la única ruta autorizada para mover unidades entre inventarios lógicos. La UI puede mostrar una proyección, pero no modificar diccionarios o listas directamente.

Las invariantes principales son conservación de unidades, capacidad no superada y disponibilidad que descuenta reservas. Los tests deben calcular totales entre proveedor/entrega, almacén, display, carrito y checkout cuando proceda. Una carga debe reconstruir contenedores y validar IDs; un producto desconocido debe generar error o estrategia explícita, no desaparecer silenciosamente.

Proveedores, pedidos y recepción El catálogo del proveedor determina productos, precio de compra y condiciones. Crear un pedido valida cantidades, coste y liquidez según diseño. La entrega no debe añadir stock directamente al almacén si el bucle exige cajas o recepción física; el estado debe avanzar por etapas observables.

La recepción transfiere unidades desde la entrega hacia el contenedor válido, conserva cantidades y registra coste una sola vez en el momento definido. Se prueban pedidos vacíos, fondos insuficientes, entrega parcial, capacidad limitada, recepción repetida y save/load en cada etapa. La UI debe mostrar estado y siguiente acción sin inventar disponibilidad.

Displays, asignación y reposición Un display tiene capacidad y, cuando el diseño lo requiere, una asignación de producto. Reponer es una transferencia atómica desde almacén u origen autorizado. No puede crear unidades ni exceder capacidad. Cambiar asignación con stock presente requiere una regla explícita: rechazar, retirar o transferir.

Las pruebas cubren display vacío, lleno, asignación distinta, stock insuficiente, reposición repetida y persistencia. En StoreInitial, los puntos de interacción y colliders deben coincidir con el visual. El mobiliario fijo puede estar en escena, pero su componente funcional debe registrarse mediante referencias o catálogo, no por búsqueda de nombres.

Clientes, perfiles y spawning El perfil del cliente define preferencias, presupuesto, paciencia u otros parámetros configurables. El spawner crea agentes bajo condiciones de tienda abierta, capacidad y seed cuando se necesita reproducibilidad. La representación visual se elige sin alterar la identidad lógica ni el comportamiento esperado.

El flujo debe avanzar por estados observables: llegada, búsqueda/browsing, evaluación, reserva, carrito, cola, checkout y salida. Un agente atascado no puede quedar indefinidamente sin timeout o recuperación. Las pruebas usan perfiles extremos, ausencia de stock, navegación bloqueada, cierre de tienda y save/load si el sistema persiste clientes activos.

Shopping, reservas y carrito La reserva protege unidades frente a otros clientes y reduce disponibilidad sin retirar prematuramente stock físico si el modelo separa ambos conceptos. Debe tener owner, cantidad, producto, origen y ciclo de vida. Cancelar, abandonar o expirar libera exactamente la reserva.

El carrito representa intención comprometida del cliente y no debe duplicar reservas al reintentar una transición. Las pruebas cubren dos clientes compitiendo por la última unidad, cancelación, paciencia agotada, cierre, cambio de display y persistencia. La UI o feedback del cliente debe distinguir falta real de stock de una reserva temporal.

Cola y estación de checkout La cola mantiene un orden determinista, normalmente FIFO, y asigna posiciones o slots de espera. Entrar dos veces debe ser imposible o idempotente. Salir de la cola libera posición y permite avanzar a los siguientes clientes sin huecos lógicos.

La estación de checkout conoce qué cliente puede ser atendido y no procesa un carrito ajeno. La escena autorada debe proporcionar anchor, orientación y espacio de navegación. Las pruebas incluyen cola vacía, múltiples clientes, abandono, cierre de tienda, estación ocupada y recuperación tras una transacción fallida.

Checkout y venta El checkout ejecuta preflight: cliente correcto, carrito/reserva válida, stock disponible, precios y estación. Después realiza un commit que consume unidades, registra ingreso, completa la reserva y produce un record de venta. Los eventos de UI, audio y VFX se emiten tras el commit.

La idempotency key impide que un retry, doble clic o reentrada procesa la misma venta. Los tests fallan cada precondition de forma aislada, repiten el comando y verifican inventario, dinero, ledger, carrito y cliente. El snapshot debe evitar que una venta completada vuelva a aplicarse al cargar. Un fallo S1 típico es dinero cobrado sin retirar stock o stock retirado sin asiento económico.

Ciclo diario y cierre El día posee fases claras: BeforeOpen, Open, Closing y Closed, con tiempo lógico y reglas de transición. Abrir habilita spawning y operaciones; el cierre detiene nuevas llegadas, resuelve clientes según política, calcula resultados y coordina autosave.

Las transiciones deben ser idempotentes y no depender únicamente de una animación o UI. Se prueban apertura repetida, cierre temprano/no permitido, clientes en cola, pedidos pendientes, autosave fallido y carga en cada fase. La hora 21:45/22:00 u otros umbrales se mantienen configurables según la especificación vigente.

Economía, dinero y ledger El dinero se representa en unidades menores enteras para evitar error de coma flotante. Toda entrada o salida tiene concepto, importe, momento e identificador cuando requiere idempotencia. El saldo no se modifica desde UI; el ledger o servicio económico aplica transacciones válidas.

Los resultados diarios derivan de ventas, costes, gastos y otros asientos, no de contadores duplicados. Las pruebas cubren sumas, límites, fondos insuficientes, repetición, descuentos/impuestos futuros si existen y equivalencia tras save/load. Un cambio de unidad monetaria o semántica exige migración de schema.

Resultados, reportes y feedback El reporte diario presenta una proyección del estado económico y operativo: ventas, margen, costes, clientes y stock relevante. Debe ser reconstruible a partir del ledger y records autoritativos. No debe modificar el resultado al abrirse ni recalcular con reglas diferentes a las usadas durante el día.

Se prueban días sin ventas, múltiples ventas, gastos, cierre repetido y localización. El reporte debe explicar siguiente acción y no ocultar pérdidas o inconsistencias. La evidencia de balance puede usar export o capturas, pero el cálculo debe validarse con tests puros.

Persistencia integrada y sidecars La captura de estado reúne sistemas en un snapshot coherente. Si existen sidecars o modelos históricos, su ownership y orden de coordinación deben estar documentados. El autosave no puede capturar algunos sistemas antes y otros después de una mutación crítica sin checkpoint coordinado.

Al restaurar, se construye un candidato completo, se valida y solo entonces se aplica. Los sistemas deben aceptar restauración en un orden que respete dependencias: catálogos/IDs, inventarios, reservas, economía, día, entidades y presentación. La evidencia más fuerte es equivalencia semántica, no igualdad textual del JSON cuando el orden no es contractual.

Operations UI, HUD, tutorial y ayuda Operations agrupa acciones de gestión sin convertirse en una segunda simulación. Cada panel consulta servicios o modelos de lectura, emite comandos y muestra resultados. El HUD presenta hora, dinero, estado y alertas con prioridad clara. Tutorial y ayuda deben poder omitirse o revisarse sin bloquear input.

Se prueban estados vacíos, carga, error, confirmación destructiva, teclado/ratón, foco, ES/EN y resoluciones. La UI no debe mostrar datos de una sesión anterior después de volver al menú. Los eventos y suscripciones se limpian al destruir o desactivar vistas.

Audio, VFX y fallbacks de presentación Los fallbacks permiten que una capacidad funcional siga siendo comprensible si falta un asset representativo, pero deben estar identificados y no competir con la presentación final. El registro runtime elige asset aprobado o fallback de forma determinista.

Cuando se sustituye un fallback, se valida que el evento siga ocurriendo una sola vez, que no aparezcan ambos visuales y que la ausencia de un clip o prefab no produzca excepción. La retirada definitiva de fallbacks se difiere hasta que la build representativa pase QA y exista rollback razonable.

Paquetes de evidencia por gate actual

Paquete de cierre de Sprint 16

- charter y estado final de work packages
- StoreInitial.unity y prefab de entorno identificados
- capturas del layout aprobado
- validación de StoreInitialSceneContext
- prueba de ausencia de shell duplicado

- corrección de input UI/mundo

HISTORICAL EVIDENCE — infraestructura automatizada retirada: - 1215 EditMode y 70 PlayMode o baseline explicada

- Golden Path post-integración
- build Windows x64 y Player.log
- defectos, known issues, cambios, documentos y handoff

El paquete debe demostrar tanto continuidad funcional como reemplazo representativo. Una build previa a la integración no sirve como build de cierre, aunque sea estable. Las capturas no sustituyen el ejecutable y las suites no sustituyen la aprobación visual.

Paquete de cierre de Sprint 17

- baseline de balance y dataset
- perfil CPU/GPU/memoria en build
- soak y campaña de siete días
- fault injection y restore
- matriz ES/EN/resoluciones/accesibilidad
- triage final y cero S0/S1
- regresión completa
- QA build y candidata H6
- documentación, auditoría y closure record

Sprint 17 debe reducir incertidumbre y defectos. Si durante estabilización aparece una necesidad de reescritura mayor, se registra como riesgo y se decide si H6 se retrasa; no se disfraza como “pulido”.

Paquete de decisión H6

- candidata inmutable con Build ID, versión, SHA y checksum
- manifest de Unity, paquetes, escenas y profile
- test reports y ejecuciones manuales
- Golden Path y campaña persistente
- Player.log y métricas de rendimiento
- defectos/deuda y known issues
- evidencia visual y de accesibilidad
- estado de documentación/trazabilidad
- acta de decisión PASS/FAIL

La decisión debe poder auditarse meses después. Los artefactos se conservan de forma que el checksum permita confirmar que el ejecutable evaluado es el mismo que el aprobado.

Manual de escalado de incidencias

Incidencia	Respuesta inmediata	Escalado
Pérdida/corrupción de save	preservar archivos, detener escritura	S0/S1, ADR/migración, build de re
Duplicación de dinero/stock	bloquear flujo, capturar estado	S1, invariante, test de regresión
Build no arranca	guardar log y entorno	S1 si gate; revisar profile/escenas
Missing reference en StoreInitial	detener cierre de escena	S1/S2 según flujo; SceneContext
Regresión visual representativa	captura y comparación	S2 o gate visual; autoría/prefab
Click UI ejecuta mundo	reproducir y bloquear acción	S2, input exclusivo
Package update rompe proyecto	revertir a baseline	ADR y spike aislado
Secreto en repo/log	revocar/rotar, preservar evidencia	S0 seguridad, limpieza e incident
Licencia desconocida	retirar de build distribuible	bloqueo legal previo a publicació
Contradicción de alcance	detener implementación	Binder + Change ID + decisión
Rendimiento cae severamente	congelar escenario y medir	S1/S2 según objetivo; profiling
Test flaky	registrar frecuencia, no rerun ciego	aislar estado/orden y corregir

Preguntas de revisión por disciplina

Producto

- ¿el resultado refuerza la fantasía de tienda física?
- ¿pertenece al gate actual?
- ¿la progresión futura se mantiene como visión y no promesa?
- ¿el jugador entiende causa y efecto?

Arquitectura

- ¿quién posee el estado?
- ¿qué assembly puede depender de cuál?
- ¿la escena declara o descubre?
- ¿el fallo es explícito?
- ¿existe rollback?

Datos

- ¿IDs y unidades son estables?
- ¿invariantes pueden romperse?
- ¿el snapshot es completo?
- ¿schema y migración están definidos?

UX

- ¿la siguiente acción es visible?
- ¿UI y mundo compiten?
- ¿hay estados vacío/error?
- ¿funciona en ES/EN y sin depender de color?

QA

- ¿el caso es reproducible?
- ¿el resultado está asociado a build?
- ¿se cubren negativos y límites?
- ¿la severidad representa el gate?

Build

- ¿SHA y versión coinciden?
- ¿escenas correctas?
- ¿TestLab excluido?
- ¿EXE y Player.log revisados?
- ¿checksum archivado?

Gobierno

- ¿el cambio tiene ID?
- ¿documentos impactados?
- ¿ADR necesario?
- ¿baseline/handoff actualizados?
- ¿se conserva la historia?

Ejemplo de sesión completa aplicada a S16-WP-004

La sesión comienza revisando que S16-WP-003 haya producido arquitectura suficiente para ubicar la entrada. Se abre el defecto de orientación de puerta y se confirma que no se intenta resolver mediante reimportación del modelo. Se consulta el criterio de StoreInitial, el TDD sobre referencias explícitas y la guía de setup sobre prefabs.

Se inspecciona StoreInitialEnvironment.prefab y se separan frame, hojas, trigger y componente funcional. Se ajustan pivotes o parents del prefab solo si el problema afecta todas las instancias; la posición final pertenece a la escena autorada. StoreInitialSceneContext o el componente correspondiente recibe referencias serializadas a hoja izquierda/derecha, trigger y anchors. Se elimina cualquier búsqueda por nombre nueva y se comprueba que no exista un script procedural reescribiendo el transform.

La validación incluye Edit Mode para orientación, Play Mode para apertura/cierre, navegación del jugador y clientes, colisiones, retorno al estado cerrado y ausencia de excepciones. Se prueban varios ciclos y entrada desde ambos lados si aplica. Después se ejecuta el test dirigido y se guarda una captura o video corto. Se actualizan defecto, tarea, cambio y evidencia. La tarea no se marca validada si todavía falta comprobarla dentro del Golden Path o build post-integración; implementación y validación permanecen separadas.

Ejemplo de sesión completa aplicada a un cambio de save

La sesión no comienza editando el DTO. Primero se crea Change ID y se describe el dato que falta y por qué debe persistir. Se revisa si pertenece a arquitectura fija, estado inicial o placement dinámico. Si el cambio altera schema, se propone ADR o decisión de migración. Se preparan fixtures de schema 1, schema 2 y archivo corrupto antes de modificar el lector.

La implementación añade el campo con ID y unidad definidos, actualiza captura y restauración, y mantiene validación en dos fases. Se prueba round trip, ausencia del campo en versión anterior, valor límite, versión futura y backup. Se comprueba que una restauración fallida no muta inventario, economía o día. Si la salida serializada cambia de orden, se compara semántica y no texto cuando el orden no es contrato.

Finalmente se actualizan Modelo de Datos, TDD, QA Matrix, migraciones de trazabilidad y Build Guide si la compatibilidad cambia. Se crea build identificada cuando el gate lo requiere. El cambio queda abierto hasta que la migración y la build estén vinculadas; compilar no equivale a cerrar.

Catálogo de antipatronos y corrección esperada

Antipatrón	Por qué falla	Corrección
Cerrar por “funciona en mi Editor”	no prueba build, configuración ni estado limpio	build externa
Usar nombres de GameObject como API	acopla runtime a jerarquía visual	referencias s
Mover datos desde la UI	duplica autoridad y rompe tests	comando a s
Mutar y después validar	deja estado parcial	preflight seg
Reintentar hasta que un test pase	oculta flakiness y contaminación	aislar estado
Reimportar assets para arreglar escena	mezcla transform fuente e instancia	corregir pref
Añadir feature futura “porque es pequeña”	abre datos, UI, save y QA ocultos	control de al
Cambiar varias dependencias a la vez	impide identificar regresión	un paquete l
Eliminar fallback antes del gate	pierde rollback y comparación	retirar tras b
Marcar deuda como defecto cerrado	borra riesgo conocido	Accepted De
Copiar texto histórico como vigente	reintroduce supuestos superados	clasificar pro
Usar porcentaje como calendario	ignora tamaño, integración y riesgo	estimación n
Bajar severidad para aprobar	maquilla el gate	reclasificar s
Hacer un commit gigantesco	dificulta review, rollback y trazabilidad	commits inte
Guardar solo el JSON final	no demuestra recuperación ni compatibilidad	fixtures, rou
Optimizar sin medir	puede empeorar claridad sin beneficio	baseline, pro
Añadir logs masivos por frame	degrada rendimiento y oculta señal	logs estructu
Confiar en una captura para validar UX	no prueba interacción, foco o flujo	recorrido ma
Crear un ADR después de implementar	convierte decisión en justificación retrospectiva	ADR antes o
Actualizar solo un documento	deja derivados contradictorios	matriz de im

Los antipatronos no son una lista de prohibiciones absolutas sin contexto. Por ejemplo, un prototipo aislado puede usar una referencia temporal; el problema aparece cuando el patrón entra en la baseline sin deuda, límite o migración. La corrección esperada debe aplicarse antes de cerrar el trabajo productivo.

Estrategia de rollback y puntos de no retorno

Todo cambio de riesgo medio o alto debe responder cómo volver atrás. El rollback puede ser de source, configuración, datos, escena o artefacto. Revertir un commit no siempre revierte un save ya migrado, un GUID cambiado o una build distribuida, por lo que los puntos de no retorno deben identificarse antes del commit.

Cambio	Rollback mínimo	Punto de no retorno
Código sin schema	revertir commit y recompilar	build publicada con dependencia externa
ProjectSettings/paquete	restaurar manifest, lock y settings	assets reserializados de forma incompatible
Escena/prefab	restaurar asset y .meta	GUID reemplazado y referencias publicadas
Store → StoreInitial	reactivar Store/fallback	save o build depende solo de nueva escena s

Cambio	Rollback mínimo	Punto de no retorno
Schema de save	lector anterior + backup/fixture	sobrescribir todos los saves sin copia compa
Catálogo/ID	restaurar asset e IDs	guardar datos con IDs reasignados
Asset representativo	volver a fallback	licencia/asset eliminado sin copia del source
Build candidata	seleccionar candidata anterior	distribución sin archivo de versión anterior
Documento normativo	restaurar versión y hash	decisiones posteriores basadas en texto erro

1. Antes de cambiar, crear baseline o identificar commit recuperable.
2. Preservar .meta, IDs, schemas y backups.
3. Definir condición que activa rollback.
4. No mezclar migración irreversible con cambios visuales no relacionados.
5. Probar rollback cuando el riesgo lo justifica.
6. Registrar qué datos o artefactos no pueden volver automáticamente.

Protocolo de retrospectiva de sprint o hito

La retrospectiva no busca asignar culpa ni producir una lista genérica. Debe convertir evidencia del sprint en cambios concretos de proceso, arquitectura o planificación. Se realiza después de congelar la baseline para no confundir reflexión con modificación del resultado evaluado.

Datos de entrada

- charter y baseline
- tiempo relativo por workstream
- WIP y bloqueos
- defectos por severidad y origen
- tests añadidos/fallidos
- builds y reintentos
- riesgos materializados
- cambios de alcance
- deuda aceptada
- incidencias de documentación/handoff

Preguntas

- ¿Qué decisión redujo más riesgo?
- ¿Qué trabajo se inició demasiado pronto?
- ¿Qué dependencia no estaba visible?
- ¿Qué evidencia se preparó demasiado tarde?
- ¿Qué defecto podría haberse prevenido con un contrato o test?
- ¿Qué documento estaba desactualizado?
- ¿Qué tarea acumuló responsabilidades incompatibles?
- ¿Qué debe conservarse sin cambios?

Salidas

- máximo tres mejoras de proceso
- deuda o ADR cuando corresponde
- ajustes de DoR/DoD
- actualización de estimación relativa
- riesgos nuevos o cerrados
- cambio de plantilla/checklist solo si aporta valor

Una mejora de retrospectiva debe tener owner y trigger. “Mejorar QA” no es accionable; “crear el test de input UI antes de integrar el próximo panel” sí lo es. No se debe reescribir el sprint cerrado para que parezca haber seguido un proceso que se decidió después.

Producción individual: disciplina sin burocracia

El proyecto se desarrolla principalmente por una persona. La documentación debe reducir carga cognitiva, no duplicarla. El criterio para formalizar es el coste de olvidar o reproducir, no el número de participantes. Una decisión de cinco minutos que altera un schema merece registro; una corrección tipográfica no necesita un ADR.

Necesidad	Registro suficiente
Recordar siguiente paso mañana	handoff breve y estado de tarea
Reproducir bug	defecto con pasos/build
Evitar repetir decisión	ADR o Change ID
Demostrar cierre	test/build/evidencia y closure
Recuperar máquina	Setup Guide y baseline
Comparar balance	dataset y métricas
Conservar idea futura	roadmap vision sin abrir backlog activo
Cambiar texto menor	commit y versión documental si aplica

La disciplina mínima viable consiste en mantener una única verdad por ámbito, estados claros, evidencia asociada y un siguiente paso explícito. No se exige completar todas las columnas de todos los libros en cada sesión. Sí se exige completar aquellas que cambian la interpretación del estado, gate, riesgo o procedencia.

Criterios para pausar, cancelar o replanificar trabajo

Condición	Decisión recomendada
Dependencia crítica no cerrada	pausar y trabajar en el predecesor
Requisito ambiguo con dos soluciones incompatibles	spike/ADR
Cambio excede sprint charter	scope control y defer
Defecto S0/S1 aparece	interrumpir trabajo no crítico
Working copy no se puede explicar	restaurar/aislar antes de continuar
Nueva build invalida baseline	reabrir gate o volver a candidata anterior
Asset sin licencia	retirar de rama distribuible
Rendimiento sin objetivo	definir métrica antes de optimizar
Migración sin rollback	no ejecutar sobre datos reales
Documentación contradice código	resolver autoridad antes de cerrar
Tarea acumula más de tres tipos de evidencia	dividir work package

Condición	Decisión recomendada
Solución requiere modificar tests para ocultar fallo	detener y revisar contrato

Cancelar no significa perder el trabajo. Se conserva aprendizaje, spike, comparación o razón de rechazo en trazabilidad cuando tiene valor futuro. Pausar debe incluir condición de reanudación; de lo contrario la tarea se convierte en WIP invisible.

Revisión de coherencia antes de publicar una baseline

1. Comprobar que versión de aplicación, Sprint y H6 coinciden en 06, 11, 12, 13, 14 y 15.
2. Comprobar que la escena objetivo y el Build Profile no usan nombres históricos contradictorios.
3. Comprobar que cifras de tests indican fecha/build y explicar cualquier diferencia.
4. Comprobar que defectos QA y deuda operativa no se presentan como el mismo contador.
5. Comprobar que documentos 00-15 tienen versión, estado, tamaño/hash y enlaces correctos.
6. Comprobar que ADR y Change IDs son únicos y que sustituciones preservan historia.
7. Comprobar que schemas, migraciones y compatibilidad coinciden entre TDD, Modelo, QA y Build Guide.
8. Comprobar que sistemas futuros permanecen VISION/DEFERRED y no aparecen como sprint activo.
9. Comprobar que cada candidata tiene SHA, manifest, checksum y Player.log.
10. Comprobar que el handoff indica cambios locales no publicados.
11. Comprobar que no existen secretos ni assets sin licencia en el paquete distribuible.
12. Ejecutar auditorías de los Excel y justificar warnings aceptados.

La auditoría global posterior a la creación de esta Guía Maestra debe usar esta lista para revisar el paquete 00-15. El resultado esperado no es necesariamente cero warnings, sino cero contradicciones sin explicar y cero fallos de integridad que impidan reproducir el estado.

Matriz de responsabilidad operativa por artefacto

Artefacto	Quién decide	Quién valida
Requisito de producto	dirección de producto mediante 00/01/02	QA y revisión de diseño
Arquitectura/ADR	responsable técnico	tests, build y revisión de consecuencias
Modelo de datos/schema	responsable de datos/persistencia	tests de invariante, migración y reco
Escena/prefab	autoría de escena y responsable técnico	QA visual, PlayMode, navegación y b
Asset/licencia	responsable de contenido	QA técnica/visual y revisión legal
Caso de prueba	QA	ejecución reproducible
Build candidata	responsable de build	QA de gate y revisión externa al Edit
Documento consolidado	owner del ámbito	revisión cruzada y auditoría
Sprint/hito	producción y gobierno	producto, técnica, QA y build como p

Aunque una sola persona desempeñe todas estas responsabilidades, debe tratarlas como revisiones distintas. Separar el “autor” del “validador” de forma mental evita aprobar una decisión únicamente porque ya se invirtió esfuerzo en implementarla. La revisión debe preguntar qué evidencia aceptaría alguien que no participó en la solución.

Protocolo de continuidad entre sesiones largas

1. Detenerse en un punto coherente: código compilable, escena guardada o experimento claramente aislado.
2. Anotar qué hipótesis se confirmó, cuál se descartó y qué incertidumbre permanece.
3. Guardar comandos, rutas, IDs y fixtures necesarios para reproducir el estado.
4. Diferenciar archivos listos para commit de archivos temporales o experimentales.
5. Actualizar el estado de la tarea solo hasta el nivel demostrado: implementación, validación o cierre.
6. Registrar el siguiente paso como una acción observable, no como “continuar”.
7. Indicar qué prueba debe ejecutarse primero al retomar y qué resultado se espera.
8. Advertir de fallbacks, flags o datos temporales activos.
9. Confirmar que no hay secretos, builds o archivos grandes añadidos accidentalmente.
10. Dejar un handoff breve incluso cuando la siguiente sesión sea propia; la memoria no es una baseline.

Este protocolo es especialmente importante durante StoreInitial, porque una sesión puede terminar con jerarquía, transforms o referencias parcialmente configurados. El handoff debe explicar qué pertenece a la escena, qué al prefab y qué sigue siendo fallback runtime. De ese modo, la siguiente sesión no corrige dos veces el mismo problema ni interpreta una solución temporal como arquitectura aprobada.

61. Cómo utilizar las autoridades especializadas 17-30

Los documentos especializados no forman una colección opcional de recomendaciones. Cada uno concentra decisiones que antes estaban repartidas por GDD, TDD, UX, QA, sprints y baselines históricas. La Guía debe usarlos como contratos de disciplina coordinados con 00-16.

61.1. Secuencia común de consulta

1. Identificar el requisito de producto en 00-05.
2. Identificar el gate, sprint o trabajo vivo en 06, 12 y 31.
3. Consultar la autoridad especializada aplicable.
4. Comprobar dependencias legales, de accesibilidad, localización, seguridad y rendimiento.
5. Crear o actualizar el ítem de producción y la trazabilidad.
6. Implementar sin exceder el alcance autorizado.
7. Ejecutar la evidencia de disciplina y la regresión transversal.
8. Actualizar catálogo, licencia, manifest o risk register cuando proceda.

61.2. Mapa de uso

Documento	Cuándo abrirlo	Salida mínima de una sesión
17 Art Bible	asset, entorno, personaje, iluminación o revisión visual	brief, asset/prefab, capturas,
18 Audio Bible	música, SFX, mezcla, feedback o captions	asset, import settings, event
19 UI Style Guide	componente, layout, tipografía, foco o HUD	prefab/componente, estados
20 Economy	precio, coste, demanda, margen o progresión	hipótesis, parámetros, simul
21 Content Catalog	alta/baja de contenido, ID, estado o cobertura	fila actualizada, ruta, owner,
22 Localization	texto, término, idioma, subtítulo o store copy	key, glosario, traducción, pse
23 Legal Register	cualquier asset, proveedor, marca o distribución	evidencia legal, obligación, c

Documento	Cuándo abrirlo	Salida mínima de una sesión
24 Steam	AppID, store, depot, demo, Playtest o release	gate, owner, artifact y decisi
25 Post-Launch	soporte, parche, rollback o incidente público	runbook, SLA/capacidad, evi
26 Privacy	dato, analytics, crash report o formulario	inventario, finalidad, minimiz
27 Security	secreto, dependencia, acceso o incidente	control, finding, playbook, ro
28 Marketing	claim, canal, campaña, prensa o creador	mensaje aprobado, media, p
29 Accessibility	barrera, setting, input, texto o claim	requisito, implementación, te
30 Performance	frame, memoria, carga, GC o hardware	escenario, captura, análisis,

62. Cómo operar 31_H6_and_Release_Readiness_Checklist.xlsx

El documento 31 es un libro de ejecución. No debe editarse como una lista de deseos ni marcarse en bloque por intuición.

62.1. Regla de estado

- NOT RUN: no existe ejecución suficiente; no equivale a fallo.
- IN PROGRESS: la prueba o el paquete de evidencia está abierto.
- PASS: criterio demostrado con evidencia verificable.
- FAIL: resultado ejecutado que incumple el criterio.
- BLOCKED: no puede ejecutarse por una dependencia explícita.
- EXCEPTION APPROVED: desviación documentada, limitada, aprobada y revisable.
- NOT APPLICABLE: exclusión justificada; nunca se usa para ocultar trabajo difícil.

62.2. Flujo de ejecución

1. Seleccionar gate y build exacta.
2. Confirmar precondiciones, commit y manifest.
3. Ejecutar el escenario descrito, sin reinterpretar el resultado esperado durante la prueba.
4. Registrar build, entorno, locale, resolución, hardware y fecha.
5. Adjuntar captura, log, archivo, hash o registro solicitado.
6. Crear defecto o excepción si no hay PASS limpio.
7. Ejecutar regresión impactada.
8. Pedir revisión y signoff humano.
9. Congelar el paquete de evidencia asociado a la decisión.

62.3. Separación de gates

Sprint 16, Sprint 17, H6, premarketing, Steam, Coming Soon, demo/Playtest, RC, lanzamiento y baseline documental tienen entradas y salidas diferentes. Un PASS en uno no se copia al siguiente; puede reutilizarse evidencia solo cuando la build, requisito y contexto permanecen válidos.

63. Cómo operar 32_Documentation_Baseline_Manifest.xlsx

El manifiesto registra qué archivos forman una baseline, de dónde proceden, qué sustituyen y cómo se verifica su integridad.

63.1. Ciclo candidato-final

1. Generar manifiesto candidato después de las actualizaciones dirigidas.
2. Auditar 00-34 y registrar findings.
3. Corregir fuentes y regenerar hashes.
4. Regenerar auditoría final.
5. Generar manifiesto definitivo cuando ningún archivo vaya a cambiar.

6. Calcular el hash del propio 32 externamente.
7. Generar checksums y ZIP.
8. Extraer el ZIP en otra ruta y verificarlo de nuevo.

63.2. Qué no hacer

- No introducir el hash final del propio XLSX dentro del archivo y esperar estabilidad.
- No deduplicar documentos históricos solo porque compartan bytes; la ruta puede aportar contexto.
- No marcar READY por presencia física.
- No reemplazar un archivo sin registrar tamaño, hash, cambio y relación de sustitución.
- No mezclar el árbol histórico con la autoridad vigente sin una carpeta y estado diferenciados.

64. Cómo operar 33_Project_Risk_Register_and_Business_Continuit

El riesgo describe un evento incierto; un incidente registra un evento ocurrido. El libro 33 comienza conservador porque los controles no deben considerarse eficaces hasta probarse.

64.1. Revisión de un riesgo

1. Validar statement de causa-evento-consecuencia.
2. Confirmar activo, fase y owner.
3. Puntuar exposición inherente sin controles.
4. Revisar controles existentes y evidencia real.
5. Elegir tratamiento: evitar, mitigar, transferir o aceptar.
6. Definir acción, fecha, criterio y owner alternativo.
7. Puntuar residual solo con controles demostrados.
8. Comparar con apetito/tolerancia.
9. Escalar si está fuera de tolerancia o bloquea gate.
10. Registrar revisión y próximo trigger.

64.2. Continuidad

Para cada capacidad crítica deben existir MTPD, RTO, RPO, estrategia, backup, runbook y ejercicio. Tener una copia no demuestra restaurabilidad. La prueba mínima restaura en un entorno limpio, compara integridad y registra duración, pérdida de datos, obstáculos y acciones correctivas.

65. Cómo operar 34_Final_Handoff_and_Project_Operations_Manual

El manual 34 gobierna la retoma y transferencia del proyecto. Se usa en tres escalas:

- **Handoff de sesión:** qué se hizo, estado, archivos, evidencia, siguiente acción y rollback.
- **Handoff de disciplina o sprint:** alcance, decisiones, backlog, builds, riesgos, deuda y aceptación del receptor.
- **Transferencia integral:** repositorio, documentación, accesos, licencias, cuentas, backups, obligaciones, runbooks y prueba de recepción.

Una transferencia no termina al entregar un ZIP o conceder acceso. Termina cuando el receptor puede restaurar, abrir, validar, identificar el siguiente trabajo y ejecutar un rollback sin depender de memoria privada.

66. Cómo ejecutar la consolidación documental v2.0

La consolidación es un proyecto con gates propios. Debe registrarse en 12 y 13 y no ejecutarse como una sucesión de reemplazos manuales sin trazabilidad.

Orden	Acción	Estado al publicar esta guía	Evidencia esperada
1	Congelar 00-34 preconsolidación	Done	snapshot, lista y checksums
2	Regenerar 12	Done	XLSX, validación y hash externo
3	Regenerar 13	Done	XLSX, validación y hash externo
4	Actualizar 14	Done	Markdown, enlaces y hash externo
5	Actualizar 15	Done al adoptar este archivo	Markdown, validación y hash externo
6	Actualizar 21	Pending	catálogo, referencias y hashes
7	Actualizar 23	Pending	legal, licenses, credits y referencias
8	Actualizar 31	Pending	32-34 existentes y controles de
9	Actualizar 33	Pending	fuentes 00-34 y riesgos de auditoría
10	Actualizar 34	Pending	rutas, versión, paquete y restore
11	Generar 32 candidato	Pending	inventario y hashes candidatos
12	Regenerar 16 candidato	Pending	findings 00-34
13	Resolver findings	Pending	cambios, evidencia y excepciones
14	Regenerar 16 final	Pending	decisión final de coherencia
15	Regenerar 32 definitivo	Pending	manifest final
16	Generar PACKAGE_MANIFEST	Pending	descripción del paquete
17	Generar CHECKSUMS_SHA256	Pending	hashes por ruta
18	Generar validation report	Pending	PASS/FAIL y excepciones
19	Crear ZIP inmutable	Pending	archivo congelado
20	Verificar extracción/restauración	Pending	reporte y hashes
21	Asociar commit/tag	Pending	commit SHA y tag
22	Activar H6 y continuidad	Pending	ejecuciones 31/33

66.1. Regla de adopción

El archivo generado fuera de la carpeta de trabajo no se considera adoptado hasta que:

- reemplaza de forma controlada la versión anterior;
- su hash se registra en 13 y, cuando corresponda, 32;
- se actualizan impactos y relaciones;
- se verifica que los documentos dependientes no quedan afirmando el estado anterior;
- la copia preconsolidación permanece intacta.

67. Cómo autorizar claims y comunicación pública

Una afirmación pública debe vincularse a una build, evidencia y aprobación. No basta con que exista como objetivo en GDD, Art Bible o plan de accesibilidad.

67.1. Matriz mínima

Claim	Evidencia mínima	Autorización
"Vertical slice jugable"	build H6 aprobada, Golden Path y signoff	02, 31
"60 FPS a 1080p"	benchmark en hardware identificado y escenario representativo	30, 31
"Compatible con mando"	recorrido completo, prompts, desconexión y regresión	05, 29
"Disponibles en español e inglés"	cobertura de strings, fonts, overflow y LQA	22, 29
Característica de accesibilidad	implementación, QA y matriz declarable de Steam	29, 31
Asset o marca de tercero	licencia, clearance, crédito y provenance	23
Fecha o ventana de lanzamiento	decisión de producción, Steam, soporte y riesgo	06, 24

Si la evidencia cambia de build o queda invalidada por una corrección, el claim vuelve a revisión.

68. Cómo integrar accesibilidad e inclusión en el trabajo diario

La accesibilidad no es un pase final. Cada feature debe revisar barreras visuales, auditivas, motoras y cognitivas desde el diseño.

1. Identificar tareas, información y acciones críticas.
2. Evitar que color, audio, precisión, tiempo o memoria sean el único canal.
3. Definir settings y defaults seguros.
4. Diseñar foco, navegación, cancelación y recuperación.
5. Probar escalas de UI/texto y extremos de localización.
6. Probar teclado, ratón y dispositivos autorizados.
7. Verificar persistencia y migración de preferencias.
8. Ejecutar QA funcional y, cuando sea posible, playtest con perfiles diversos.
9. Registrar qué está implementado, validado y declarable.
10. No usar WCAG, XAG o Steam como sello automático de conformidad; son referencias y matrices de verificación.

69. Cómo medir y optimizar rendimiento

La optimización comienza con un escenario reproducible y una pregunta. Debe evitarse cambiar calidad visual, arquitectura o comportamiento por una lectura aislada.

69.1. Registro mínimo de profiling

- build y commit;
- Unity, URP y plataforma;
- hardware, driver, resolución, modo de ventana, VSync y frame cap;
- escena, seed, clientes, inventario y duración;
- CPU/GPU frame time, percentiles, memoria, GC y cargas;
- captura o profiler data;
- hipótesis y cambio aislado;
- comparación antes/después;
- regresión visual, funcional y accesible;
- decisión y deuda restante.

Los objetivos históricos —60 FPS a 1080p, hasta ocho clientes, carga del slice menor de cinco segundos, guardado/cierre menor de dos segundos y allocations próximas a cero en estado estable— siguen siendo objetivos pendientes de validación contra hardware aprobado.

70. Cómo coordinar legal, privacidad, seguridad y continuidad

Estas disciplinas se solapan, pero no son intercambiables:

- **23 Legal:** derecho a usar, distribuir, acreditar y cumplir obligaciones.
- **26 Privacy:** qué datos pueden tratarse, para qué, durante cuánto tiempo y con qué transparencia.
- **27 Security:** cómo se protegen cuentas, secretos, builds, dependencias y respuesta técnica.
- **33 Continuity:** cómo se mantiene o recupera la operación tras una interrupción.

Ante un nuevo proveedor o servicio, debe abrirse una revisión única que cubra las cuatro perspectivas. La salida mínima incluye owner, contrato/licencia, datos, acceso, secretos, dependencia, backup/exportación, exit plan, incident contact y evidencia.

71. Cómo validar el paquete documental final

La validación combina controles estructurales y revisión semántica.

71.1. Controles automáticos

- archivos esperados y presentes;
- nombre, extensión, tamaño y hash;
- Excel sin errores de fórmula;
- Markdown con front matter, encabezados, tablas y bloques equilibrados;
- enlaces relativos y rutas;
- duplicados y archivos inesperados;
- manifiesto/checksum coherentes;
- ZIP extraíble y verificable.

71.2. Controles humanos

- autoridad y estado correctos;
- ausencia de claims no demostrados;
- relaciones y sustituciones coherentes;
- métricas con fuente y fecha;
- límites de alcance preservados;
- instrucciones ejecutables por otra persona;
- findings BLOCKER/HIGH resueltos o exceptuados formalmente;
- diferencias entre historial, objetivo, implementación y evidencia claramente etiquetadas.

72. Rutas de onboarding por rol

72.1. Producción y dirección

Leer 00, 02, 06, 12-16, 28, 31-34. Salida: entender alcance, gates, estado, riesgos y siguiente decisión.

72.2. Ingeniería

Leer 00-05, 09-11, 13-14, 26-27, 29-30, 31, 33-34. Salida: entorno reproducible, arquitectura, DoD, pruebas y recovery.

72.3. Arte, audio y UI

Leer 00-05, 10, 14, 17-19, 21-23, 28-31 y 34. Salida: brief, pipeline, provenance, aceptación y entrega.

72.4. QA y release

Leer 02, 05-08, 11-16, 22-31, 33-34. Salida: matriz, build, evidencia, defectos, gates y signoff.

72.5. Marketing, comunidad y soporte

Leer 00-02, 14-15, 17, 19, 22-29, 31, 33-34. Salida: mensajes autorizados, media aprobada, calendario condicionado y escalation path.

73. Cómo conservar y usar la documentación histórica

El árbol histórico contiene 745 archivos procedentes de baselines oficiales, fuentes Markdown, PDFs publicados, Excel, ADR, sprints, builds, QA, handoffs y material de trabajo. Se conserva por cinco motivos:

1. demostrar genealogía y decisiones;
2. recuperar detalle que una revisión breve pudo omitir;
3. verificar qué afirmación era válida en un momento;
4. reconstruir artefactos o procesos perdidos;
5. auditar sustituciones y evitar reintroducir decisiones descartadas.

La historia no se edita para que coincida con el presente. Se clasifica como autoridad sustituida, soporte, evidencia o borrador; se cita por ruta y hash; y se consulta después de la baseline actual, salvo investigación genealógica o recuperación.

74. Matriz de decisión sobre qué artefacto actualizar

Cambio observado	Actualizar siempre	Actualizar según impacto
Regla de producto	01, 13	02-08, 12, especializados afectados
Alcance H6	02, 06, 13, 31	01, 07-08, 12, 16, 33
Arquitectura	03, ADR/13	04, 09-11, 29-30, 34
Schema/save	04, 13	03, 07-08, 11-12, 26-27, 31, 33
Flujo UX/UI	05, 13, 19	07-08, 22, 29, 31
Sprint/gate	06, 12, 13	02, 07-08, 31, 33, 34
Asset/contenido	21, 23	17-20, 22, 28-31
Proveedor/servicio	23, 26, 27, 33	10-11, 24-25, 28, 34
Claim/campaña	28, 13	17, 19, 22-24, 29-31
Build candidata	08, 11-13, 31	24-25, 30, 32-34
Documento regenerado	13	12, 14-16, 31-34 y dependientes
Finding de auditoría	16, 13	fuelle responsable, 31 y 33

75. Criterios de aceptación de esta Guía Maestra

Esta revisión se considera aceptable cuando:

- gobierna 00-34 y no solo 00-14;
- conserva procedimientos útiles de las guías v0.3-v0.6;
- distingue producto, implementación, validación, release y documentación;
- describe el estado v2.0 sin declarar H6 ni release aprobados;
- integra 31, 32, 33 y 34 como herramientas operativas;
- mantiene rutas por rol, tarea, gate e incidente;
- explica la secuencia de consolidación y los artefactos finales;
- enlaza todos los documentos actuales por nombre canónico;
- registra tamaños y hashes externos disponibles;
- no borra historia ni presenta fuentes sustituidas como autoridad vigente;
- puede ser seguida por otra persona sin acceder al historial del chat.

Anexo A. Inventario de la baseline documental utilizada

| --- | --- | --- | --- | --- | --- | | 15 | 15_Guia_Maestra.md | Guía operativa | 2.0 | UPDATED v2.0 |
SELF AFTER EXPORT | EXTERNAL_AFTER_EXPORT |

A.1. Interpretación del inventario

- READY / CURRENT CANDIDATE significa que no tiene una actualización dirigida abierta antes de la auditoría, pero puede cambiar por un finding propagado.
- REGENERATED o UPDATED v2.0 identifica una salida nueva de la consolidación actual.
- DIRECTED UPDATE PENDING exige una revisión concreta aunque el archivo exista.
- CANDIDATE – FINALIZE LATER indica que el manifiesto debe regenerarse después de establecer todos los demás archivos.
- Los hashes del propio documento que los contiene se registran externamente para evitar autorreferencia inestable.

Anexo B. Evolución de las Guías Maestras históricas

Versión	Estado que representaba	Aporte conservado
v0.3	preproducción reiniciada	principios, fases, jerarquía y disciplina documental
v0.4	fundación inicial real	separación entre intención e implementación
v0.5	Sprints 0-5 cerrados	baseline jugable, arquitectura, QA y handoff
v0.6	Sprints 0-15 y Sprint 16 en curso	estado técnico, StoreInitial, pruebas y continuidad entre sesiones
1.0	consolidación inicial 00-15	manual operativo extenso y procedimientos end-to-end
2.0	baseline candidata 00-34	operación integral, cierre documental, readiness, riesgo y handoff

Anexo C. Fuentes de consolidación

- Guías Maestras v0.3, v0.4, v0.5, v0.6 y 1.0.
- Baseline actual completa 00-34.
- Excel Maestro de Producción 12 regenerado.
- Trazabilidad y Control de Cambios 13 regenerado.
- Project Binder 14 actualizado.
- QA Matrix 08 y H6/Release Readiness 31.
- Manifiesto candidato 32 y registro de riesgos/continuidad 33.
- Final Handoff and Project Operations Manual 34.
- Current Project Handoff, Current Project Baseline Record, Sprint History Summary y closure ledger.
- Sprint 16 Two-Phase Charter, timeline, status, authoring plan y acceptance matrix.
- Sprint 17 Opening Brief.
- Registro consolidado de ADR, commits, builds, defectos, pruebas y migraciones.
- Baselines oficiales v0.3-v0.6, sus PDFs, fuentes Markdown, manifests, checksums e informes de validación.
- 745 archivos históricos inventariados con 643 hashes únicos y 98 grupos de duplicados preservados.

Anexo D. Próxima revisión obligatoria

La siguiente revisión de esta guía se realizará después de:

1. actualizar 21 y 23;
2. actualizar 31, 33 y 34;
3. generar 32 candidato;
4. regenerar 16 y resolver findings;
5. fijar nombres, rutas, hashes, commit y tag definitivos.

La revisión final sustituirá 2.0 por la versión aprobada de baseline y actualizará el estado real de H6, continuidad y paquete inmutable sin anticipar resultados.

66. Operación de la remediación Sprint17_Phase1

Estado operativo vigente tras W0

Elemento	Estado vigente
Baseline de entrada	main@efbc1a885a1d6819f764ec8cff8a465ad1986661 / aplicación 0.0.26
W0 documental	COMPLETED / DOCUMENTAL PASS
Decisiones funcionales abiertas	0
Sprint17_Phase1	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
H6	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
Vertical Slice	ACCEPTED / IMPLEMENTED / VALIDATED / CLOSED

Decisiones formalizadas

ADR	DEC	Decisión cerrada
ADR-0074	DEC-01	Día H6 configurable de 300 s; velocidades visibles x0,5, x1, x2 y x4; PauseService in
ADR-0075	DEC-02	SimulationClock / StoreDay es la única autoridad de tiempo de negocio
ADR-0076	DEC-03	Guardado manual solo desde pausa en BeforeOpen, Closed o Results, sin mutacione
ADR-0077	DEC-04	Checkout funcional obligatorio para abrir; no retirar el último durante Open
ADR-0078	DEC-05	Máximo H6 configurable de 8 clientes activos; activo = no Despawned
ADR-0079	DEC-06	La solicitud reserva fondos; la recepción descuenta caja, registra SupplierReceiving
ADR-0080	DEC-07	Process All es todo-o-nada sobre órdenes independientes, reserva el total y crea un
ADR-0081	DEC-08	Displays monoproducto hasta H6; cantidades, retirada, retorno y limpieza de vacío ob
ADR-0082	DEC-09	Management conserva detalle del día actual y los dos completados anteriores; días m
ADR-0083	DEC-10	Al cerrar el día 7 se aplica una vez el 10 % de max(0, resultado bruto técnico se
ADR-0084	DEC-11	Wall occlusion desactivada mediante feature flag prioritaria; toggle oculto; valores an
ADR-0085	DEC-12	Contrato de pivote base-centro mediante wrappers o GroundAnchor, con migración/c

El siguiente identificador ADR disponible queda reservado como ADR-0088.

Pruebas de caracterización previas a las olas

Grupo	IDs	Ola protegida	Cobertura mínima
Tiempo y pausa	CHAR-TIM-001 a CHAR-TIM-008	W1	reloj, velocidades, pausa anidada, H
Clientes y checkout	CHAR-CUS-001 a CHAR-CUS-009	W2	ocho activos, checkout obligatorio, H
Pedidos y economía	CHAR-ODR-001 a CHAR-ODR-010	W3	reserva de fondos, Process All, recei
Displays	CHAR-DSP-001 a CHAR-DSP-006	W4	asignación única, cantidades, retorn
Guardado	CHAR-SAV-001 a CHAR-SAV-008	W5	estados seguros, mutaciones pendie
Management	CHAR-MGT-001 a CHAR-MGT-008	W6	día actual + 2, DailySummary, lifetin
Autoría y settings	CHAR-ART-001 a CHAR-ART-007	W7	occlusion false, migración de prefer
Regresión integral	CHAR-REG-001 a CHAR-REG-010	W8	siete días, build externa, logs, save/

Los casos de caracterización fueron ejecutados y validados para cerrar W1-W8 y la Vertical Slice 0.0.26. Cualquier cambio futuro deberá preservar o volver a validar esta cobertura.

Secuencia operativa obligatoria

1. Conservar una copia recuperable de la baseline efbc1a8 / 0.0.26.
2. Ejecutar cada grupo de caracterización antes de la ola correspondiente.
3. Aplicar W1 a W7 en orden de dependencias, sin mezclar decisiones funcionales nuevas.
4. Actualizar contratos, trazabilidad, riesgos y QA cuando una implementación difiera de la decisión.
5. Ejecutar W8 sobre build externa y campaña persistente de siete días.
6. Mantener H6 y Vertical Slice bloqueados hasta signoff formal en el libro 31.

Regla de cierre y no propagación

- W0–W8 están completadas, validadas y cerradas en la baseline 0.0.26.
- Sprint17_Phase1 y W8 están completadas, validadas y cerradas.
- H6 y la Vertical Slice disponen de aceptación propia y están cerrados sobre 0.0.26.
- La Vertical Slice queda cerrada; los sistemas Post-H6 se gestionarán como evolución posterior mediante control de cambios.
-

67. Decisión final e implementación de modificaciones diferenciadas S17

Tras el cierre W0–W8 / PASS, se aprueba la implementación integral de S17-MOD-002 a S17-MOD-010. S17-MOD-001 queda absorbida por el contrato de reloj continuo de S17-MOD-007.

Decisión	Estado candidato	Fuente normativa
Visitas sin producto visible	IMPLEMENTED / PENDING VALIDATION	S17-MOD-002
Una visita por DeliveryRun	IMPLEMENTED / PENDING VALIDATION	S17-MOD-003
Inventario disponible solo tras recepción	IMPLEMENTED / PENDING VALIDATION	S17-MOD-004
LOD persistente y caja proporcionada	IMPLEMENTED / PENDING VALIDATION	S17-MOD-005
Hora HH:mm	IMPLEMENTED / PENDING VALIDATION	ADR-0086
Reloj continuo salvo pausa	IMPLEMENTED / PENDING VALIDATION	ADR-0086
Aperturas múltiples 08:00–22:00	IMPLEMENTED / PENDING VALIDATION	ADR-0087
Grounding mundial sin offsets por prefab	IMPLEMENTED / PENDING VALIDATION	S17-MOD-009 / ADR-008
Staff orientado por anchors	IMPLEMENTED / PENDING VALIDATION	S17-MOD-010

Los nuevos ADR vigentes son:

- ADR-0086: reloj global continuo y proyección de 24 horas;
- ADR-0087: ventana comercial y aperturas múltiples.

El siguiente identificador ADR disponible es ADR-0088. El estado de esta sección no sustituye la validación en Unity; la candidata deberá superar la regresión completa W1–W8 más los nuevos casos S17-MOD antes de declararse PASS.